淡江大學電機工程學系人工智慧機器人碩士班

碩士論文

指導教授：李世安博士

基於 RISC-V 開源處理器架構的 Avalon 匯流排橋接器之設計與實現

Design and Implementation of Avalon Bus Bridge Based on RISC-V Open-Source Processor Architecture

研究生：葉玟霖　撰

中華民國 112 年 1 月

誌謝

首先，最感謝的是我的家人，給予我經濟上的支持，讓我在 25 年的生活中不曾為錢煩惱，成長和求學的過程中跌跌撞撞，也無條件地包容我，終於到了人生的下一個里程碑，接下來換我來照顧你們。

感謝世安老師，亦師亦友的陪伴與教導，你是我看過最有耐心的老師，常常坐在學生旁一步一步帶領著我們；每當我懷疑自己的能力時，總是跟我說沒關係試試看，也感謝讓我有機會當助教，透過教學讓我能更認清我自己，發現自己的不足，從以前什麼都不懂，到現在能夠跟老師討論更深的架構細節，真的很感謝老師願意花時間栽培我們，讓我們成長茁壯。

感謝我的兒時玩伴們，宋浩宇、黃任遠、宋昱璋，蔡博岳、黃彥翔，當我一個人在淡水感到寂寞時，總會揪我一起出來玩。

感謝高大衛同學，謝謝你跟我分享了很多心理話，讓主耶穌給了我第二次機會來認識祂。

感謝陳建凱學長，成為我的忘年之交，帶我用不同的角度來看事情，讓我接觸到同齡人不曾接觸的人、事、物，與上市公司面談、跟創投接觸，一心想要帶著我們淡江的學弟創業，不管最後有沒有成功，這都是難得的經驗。

最後，感謝一路上幫助和陪伴我的人，我能走到今天不是靠我一個人的力量，由衷的感謝你們。

謹以此論文獻給所有愛我的人與我愛的人

葉玟霖 謹誌

民國 112 年 冬 于淡水

論文名稱：基於 RISC-V 開源處理器架構的 Avalon 匯流排橋接器之設計與實現　頁數：87

校系(所)組別：淡江大學　電機工程學系人工智慧機器人碩士班

畢業時間及提要別：　111 學年度第 1 學期　碩士學位論文提要

研究生：葉玟霖　指導教授：李世安　博士

論文提要內容：

本論文目的是在 Intel 公司推出的 DE 系列 FPGA 開發平台上，實現一個 RISC-V 系統。首先本論文設計一個 Picorv32 處理器與 Avalon bus 連接的橋接器，使 Picorv32 處理器能夠透過 Avalon 匯流排存取 Qsys 系統上的記憶體和周邊(Peripheral)，並且在 DE2-115 開發平台上對此 RISC-V 系統進行模擬、驗證和展示。

本論文包含了軟硬體的設計與開發流程，在軟體上，有編譯器工具鏈和指令集的選用、連結器腳本的修改、執行檔的格式轉換、記憶體初始化和燒錄；在硬體上，設計了 Picorv32 處理器與 Avalon 匯流排之間的橋接器，並對橋接器做出完整的時序分析和波形驗證，同時也設計了基於硬體架構的程式下載器，用來幫助下載程式到記憶體中。

在論文的最後比較了 Picorv32 和 Nios II 處理器的性能測試以及硬體上的邏輯資源消耗，從實驗結果證明 Picorv32 處理器的設計架構是優於 Nios II/e 處理器架構，將有機會取代 Nios II/e 處理器成為系統的核心處理器。

關鍵字：

Picorv32、RISC-V、FPGA、Avalon 匯流排、橋接器、Nios II

＊依本校個人資料管理規範，本表單各項個人資料僅作為業務處理使用，並於保存期限屆滿後，逕行銷毀。

Title of Thesis： Total pages:87

Design and Implementation of Avalon Bus Bridge on RISC-V Open-Source Processor Architecture

Key word: Picorv32, RISC-V, FPGA , Avalon bus, Bridge, Nios II

Name of Institute:
Department of Electrical and Computer Engineering, Tamkang University

Graduate date: January 2023 Degree conferred: Master

Name of student: WEN-LIN YEH 葉玟霖
Advisor: Dr. Shih-An Li 李世安 博士

Abstract:

The purpose of this thesis is to implement a RISC-V system on the DE series FPGA development platform released by Intel Corporation. Firstly, this thesis designs a bridge connector that connects the Picorv32 processor with the Avalon bus, enabling the Picorv32 processor to access memory and peripherals in the Qsys system through the Avalon bus. The design is then simulated, validated and demonstrated on the DE2-115 development platform.

The thesis includes both hardware and software design and development processes. On the software side, there are compiler tool chains, instruction set selection, modification of linker scripts, format conversion of executables, memory initialization and burning. On the hardware side, the design of the bridge connector between the Picorv32 processor and the Avalon bus is presented, with a complete timing analysis and waveform verification, and a hardware-based program downloader is also designed to help download programs into memory.

At the end of the thesis, a performance comparison is made between the Picorv32 and Nios II processors, as well as a logic resource consumption comparison in hardware. The experimental results prove that the design architecture of the Picorv32 processor is superior to that of the Nios II/e processor, and has the potential to replace the Nios II/e processor as the core processor of the system.

According to “TKU Personal Information Management Policy Declaration“, the personal information collected on this form is limited to this application only. This form will be destroyed directly over the deadline of reservations.

目錄

目錄 III
圖目錄 V
表目錄 IX
中英文對照表 X
第一章 緒論 1
1.1 研究背景 1
1.2 研究動機 1
1.3 研究目的 3
1.4 論文組織 4
第二章 實驗平台與開發環境 6
2.1 實驗平台 6
2.2 系統架構 9
第三章 硬體設計 12
3.1 橋接器設計 12
3.2 Picorv32 處理器介紹和參數修改 24
3.3 SOPC 設計 27
3.4 Debug module 設計 33
第四章 軟體編譯和燒錄流程 37
4.1 軟體編譯流程 37
4.2 周邊裝置記憶體映射地址 38
4.3 連結器腳本修改 40
4.4 執行檔案的格式轉換 44

4.5 程式下載和記憶體初始化 48
第五章 實驗結果 56
5.1 實驗一：基礎 IO 測試和 Demo 56
5.2 實驗二：JTAG UART Demo 65
5.3 實驗三：使用 Nios II JTAG Debug module 下載程式 69
5.4 處理器資源消耗和性能比較 82
第六章 結論與未來展望 84
6.1 結論 84
6.2 未來展望 85
參考文獻 86

圖目錄

圖 1.1、T-Core e203 RISC-V SoC 實驗架構 3
圖 2.1、DE2-115 教育開發平台[3] 6
圖 2.2、系統架構 11
圖 3.1、橋接器架構 13
圖 3.2、橋接器輸入輸出和位元寬度定義 14
圖 3.3、Avalon Memory-Mapped 介面典型讀寫時序[6] 16
圖 3.4、指令讀取時序 17
圖 3.5、SignalTap II 指令讀取時序 18
圖 3.6、資料讀寫時序 19
圖 3.7、SignalTap II 資料讀取時序 19
圖 3.8、SignalTap II 資料寫入時序 20
圖 3.9、橋接器內部三塊輸出邏輯 21
圖 3.10、橋接器內部邏輯 Verilog Code 21
圖 3.11、Avalon Instruction Master Output Logic RTL View 22
圖 3.12、Avalon Data Master Output Logic RTL View 23
圖 3.13、Picorv32 Native Memory Interface Output Logic RTL View 24
圖 3.14、Memory layout 27
圖 3.15、打包 Picorv32 和橋接器 28
圖 3.16、分析檔案 29
圖 3.17、確認信號 29
圖 3.18、Instruction Master Associated Reset 30
圖 3.19、Data Master Associated Reset 30

圖 3.20、Qsys IP Library 31
圖 3.21、SOPC interconnection 32
圖 3.22、分配記憶體映射空間 33
圖 3.23、FSM 狀態圖 36
圖 4.1、軟體編譯流程[13] 38
圖 4.2、sopcinfo 轉檔流程 39
圖 4.3、sopcinfo 轉檔操作 39
圖 4.4、header file 部分資訊 40
圖 4.5、連結器和連結器腳本 41
圖 4.6、修改 riscv.ld 程式進入點 42
圖 4.7、修改 sections.lds 43
圖 4.8、程式進入點和起始地址 44
圖 4.9、ELF 檔案格式[17] 45
圖 4.10、ELF 檔案格式轉換 45
圖 4.11、General Record Format 46
圖 4.12、Hex File 範例 47
圖 4.13、Extended Linear Address Record 47
圖 4.14、Hex File 初始化記憶體 49
圖 4.15、Hex File 初始化 On-Chip Memory 50
圖 4.16、更改 Hex File 資料地址 50
圖 4.17、Debug module 韌體程式流程圖 52
圖 4.18、使用 Nios II EDS 燒錄流程 53
圖 4.19、Nios II 程式下載器 54

圖 5.1、IO 測試之硬體 SOPC 架構 56
圖 5.2、基礎 IO 測試的軟硬體開發流程 58
圖 5.3、Demo 程式流程圖 59
圖 5.4、周邊元件基底地址和副函式定義 60
圖 5.5、IO Demo 主函式 61
圖 5.6、Demo 範例 62
圖 5.7、SW_BASE 讀取波形 63
圖 5.8、HEX7_BASE 寫入波形 63
圖 5.9、HEX6_BASE 寫入波形 64
圖 5.10、HEX5_BASE 寫入波形 64
圖 5.11、HEX4_BASE 寫入波形 65
圖 5.12、JTAG UART Demo 系統架構 65
圖 5.13、JTAG UART Demo 軟硬體開發流程 66
圖 5.14、JTAG UART 基底地址和副函式定義 67
圖 5.15、JTAG UART Demo 主函式 68
圖 5.16、nios2-terminal 顯示 JTAG UART 傳輸文字 69
圖 5.17、實驗流程 70
圖 5.18、實驗三實驗步驟 71
圖 5.19、程式下載器 72
圖 5.20、配置 FPGA 成為程式下載器 73
圖 5.21、讀取 SDRAM 數值並以 JTAG UART 傳輸 74
圖 5.22、下載 a.elf 執行檔到 SDRAM 75
圖 5.23、目標系統架構 75

圖 5.24、設定 SDRAM 大小.....76
圖 5.25、設定 SDRAM 時序控制參數.....77
圖 5.26、設定系統時脈 c0.....77
圖 5.27、設定 SDRAM 時脈 c1.....78
圖 5.28、目標系統的通訊架構.....79
圖 5.29、配置 FPGA 成為目標系統.....80
圖 5.30、nios2-terminal 顯示 JTAG UART 資料.....81
圖 5.31、objdump a.elf 執行檔.....82

表目錄

表 2.1、Cyclone IV EP4CE115 FPGA 晶片資源 7
表 2.2、DE2-115 外部記憶體資源 7
表 2.3、硬體開發工具 8
表 3.1、Picorv32 原生記憶體介面信號說明 14
表 3.2、Avalon Memory-Mapped 介面信號說明 16
表 3.3、指令集說明 25
表 3.4、參數說明 26
表 3.5、各檔案對應電路 28
表 4.1、RECORD TYPE 46
表 4.2、程式下載方式比較 55
表 5.1、共陽極七段顯示器解碼 63
表 5.2、JTAG UART 的暫存器說明 67
表 5.3、處理器資源消耗比較 83
表 5.4、處理器性能比較 83

中英文對照表

英文	中文
Assembler	組譯器
Base Address	基底地址
Bridge	橋接
Combinational	組合
Compiler Toolchain	編譯器工具鏈
Component	元件
Debug Module	除錯模組
Detection	偵測
Embedded System	嵌入式系統
Field Programmable Gate Array, FPGA	現場可程式化邏輯閘陣列
Finite-State Machine	有限狀態機
Initial	初始化
Linker	連結器
Loader	載入器
Logic Cell	邏輯單元
Logic Element	邏輯元件
Object file	目的檔
Performance Benchmarks	性能基準
Peripheral	周邊
Programmable	可程式化
Register	暫存器
Relocation	重定位
Sequential Detector	序列偵測器
Shift Register	移位暫存器
System On a Chip, SoC	單晶片系統
System On Programmable Chip, SOPC	可程式化單晶片系統

第一章 緒論

1.1 研究背景

美國總統拜登(Joseph Robinette Biden Jr.)於 2022 年 8 月 9 日簽訂晶片法案，限制了中國大陸的晶片製造和高階晶片進口，在美國晶片法案下的科技冷戰中[1]，迫使中國開始自主研發晶片的設計和製造，因此 RISC-V，一個由美國加州大學柏克萊分校(UC Berkeley)於 2010 年研發的開源指令集架構，迅速地在海峽兩岸竄紅，不僅指令集開源，任何人都可以實作此指令集架構，且它的架構彈性，相對於 ARM 和 x86 指令集少去了歷史上的包袱，支援眾多擴增指令集以及客製化指令；RISC-V 基金會於 2015 年 8 月成立，現今設立於瑞士，在政治態度上保持絕對中立，且承諾將不收取授權金，以上述觀點來看，在 SoC(System On a Chip)的設計上選用以 RISC-V 指令集架構之處理器無疑是最佳選擇。

1.2 研究動機

在 2022 年 8 月 26 日，本人參加了由教育部補助的智慧晶片系統與應用課程推廣計畫-種子教師培訓營，授課講師是台灣科技大學資工系的劉一宇老師和友晶科技的范振能資深經理，友晶科技在 T-Core MAX10 FPGA(Field Programmable Gate Array)開發平台上移植了蜂鳥 e203 RISC-V SoC[2]，由劉一宇老師及助教在課堂上演示了一些 Demo，如控制 GPIO 點亮 LED 燈、UART 傳輸、增加客製化 dot 指令等等。

在培訓營的實做的過程中，發現了一些很麻煩的問題，實驗架構如圖 1.1 所示，首先整個實驗橫跨三個作業系統，分別是 Windows10, WSL(Windows Subsystem for Linux)和 VirtualBox Ubuntu 18.04，在

Windows10 作業系統中使用 Quartus Prime18.1 來合成蜂鳥 e203 RISC-V SoC，並配置到 FPGA 上，在 WSL 作業系統中安裝 RISC-V 的 Compiler Toolchain 用來開發 RISC-V 軟體程式，在 VirtualBox Ubuntu 18.04 作業系統中，友晶科技提供了 RISC-V 的程式下載工具，幫助編譯好的 RISC-V 程式下載到 T-Core 開發平台上的 QSPI Flash，由於下載工具是在虛擬機上執行，降低了執行效率，導致程式下載過程接近 50 秒左右，綜觀整個實驗錯綜複雜且冗長，部分檔案須在三個作業系統中來回搬動，在 T-Core 開發板的硬體設計上，由於要配置 FPGA 又要下載 RISC-V 程式到 Flash 中，因此拉了兩組 JTAG(Joint Test Action Group)到板子上，由指撥開關來決定要做哪種操作，對於使用者來講也是一件麻煩事，尤其微型指撥開關非常小不易操作，要 Demo UART 印出 Debug 資料時還要額外指定 GPIO 再透過 USB to TTL Converter 模組來幫忙連接到 Host PC 來顯示，無疑是增加硬體了成本。

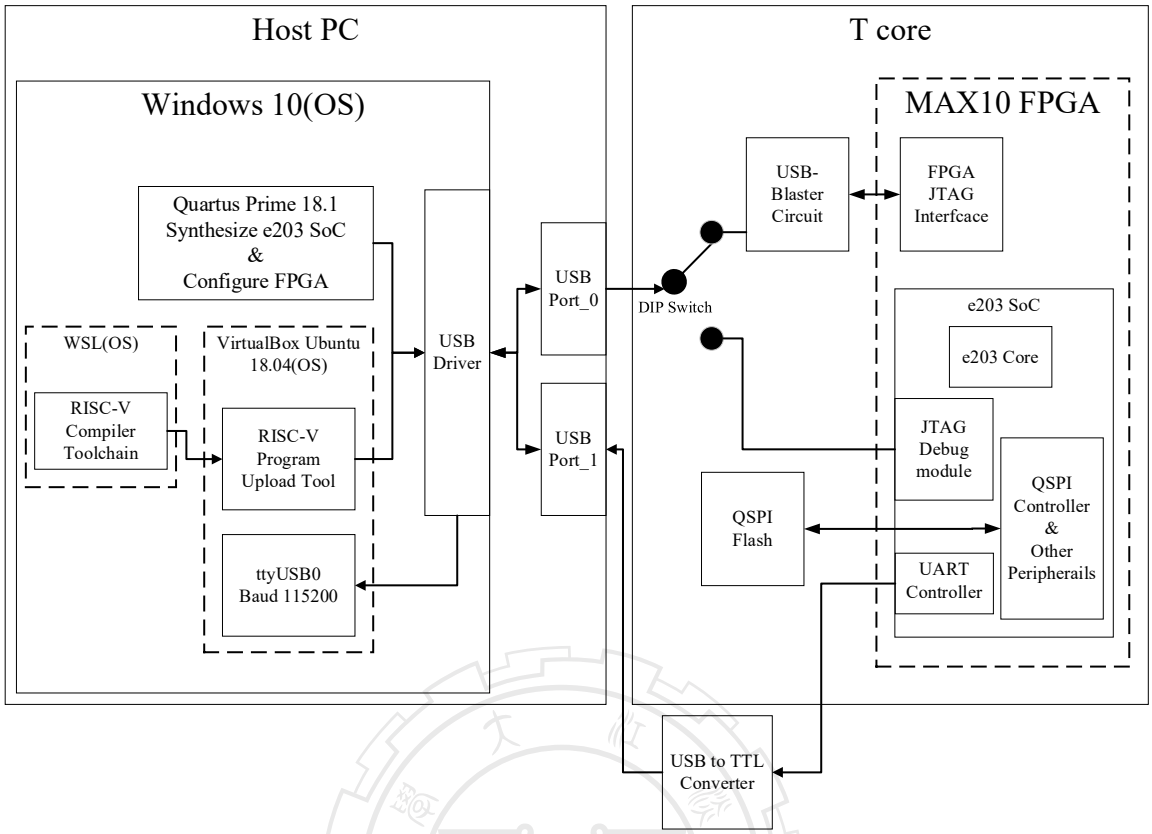


圖 1.1、T-Core e203 RISC-V SoC 實驗架構

在 e203 RISC-V SoC 的設計上由於內部的系統匯流排(System Bus)架構並不是採用 Avalon 匯流排架構，因此在增加周邊元件或是周邊控制器時變得很困難，要手動處理內部連接(Interconnection)，若是採用 Avalon 匯流排架構就可以透過 Qsys 工具輕鬆地增加周邊元件。

本論文在設計橋接器和 Avalon 系統架構之 SOPC 時也試圖解決以上開發的不便，使用本論文的橋接器設計搭配 Avalon 系統架構，將有機會縮短開發流程和減少硬體材料成本，將在後續章節闡述設計過程和結果。

1.3 研究目的

由於不同處理器的記憶體傳輸介面都不太一樣，為了繼續沿用以

Avalon 匯流排介面開發的周邊 IP，以及繼續使用 Qsys 工具來打造我們的 SOPC，RISC-V 處理器與 Avalon 匯流排的橋接是勢必會遇到的課題，因此本論文將以 Avalon 匯流排橋接器為主軸進行撰寫。

1.4 論文組織

本論文的架構分為六個章節，以下針對每個章節做簡要說明。

第一章：緒論

介紹本論文選用 RISC-V 指令集架構處理器的原因，以及使用 Avalon 系統匯流排架構的好處，並以友晶科技在 T-Core 開發平台上 Demo 的 RISC-V SoC 解決方案中點出目前在硬體移植上的限制和使用者在程式開發上的不便。

第二章：實驗平台與開發環境

介紹實驗過程中使用到的硬體設備、軟硬體的開發環境和工具，以及講述整體的實驗架構。

第三章：硬體設計

介紹橋接器的硬體設計與時序分析、基於硬體的程式下載器設計，示範 Picorv32 處理器與 Avalon 系統匯流排如何進行橋接，以及如何搭建整個 SOPC 系統。

第四章：軟體編譯和燒錄流程

介紹如何使用 RISC-V GNU Compiler Toolchain 編譯出可執行檔，並透過不同工具和方法下載程式到記憶體中。

第五章：實驗結果

對移植整合後的 SOPC 進行 LED、七段顯示器、指撥開關、JTAG UART、SDRAM 等 IO 和記憶體元件進行驗證和展示，並對 Picorv32 和 Nios II 處理器做出資源消耗和性能比較。

第六章：結論與未來展望

總結本論文的貢獻，對此論文提出能夠改進的和持續研究的項目。

第二章 實驗平台與開發環境

本章節將介紹實驗中所使用的硬體設備和軟體開發工具，並講述整個實驗的系統架構。

2.1 實驗平台

2.1.1 FPGA 開發板

本論文使用到的 FPGA 開發板為友晶科技的 DE2-115 教育開發平台如圖 2.1 所示，其 FPGA 晶片型號為 Intel Cyclone IV EP4CE115，邏輯資源豐富，且開發板上搭配了眾多周邊 IO 裝置以及多媒體影音功能，讓開發者能更便捷的開發與驗證；FPGA 晶片及開發板外部記憶體資源如表 2.1 和表 2.2 所示，開發板上還有許多 IO 設備，本小節僅列重要資源。

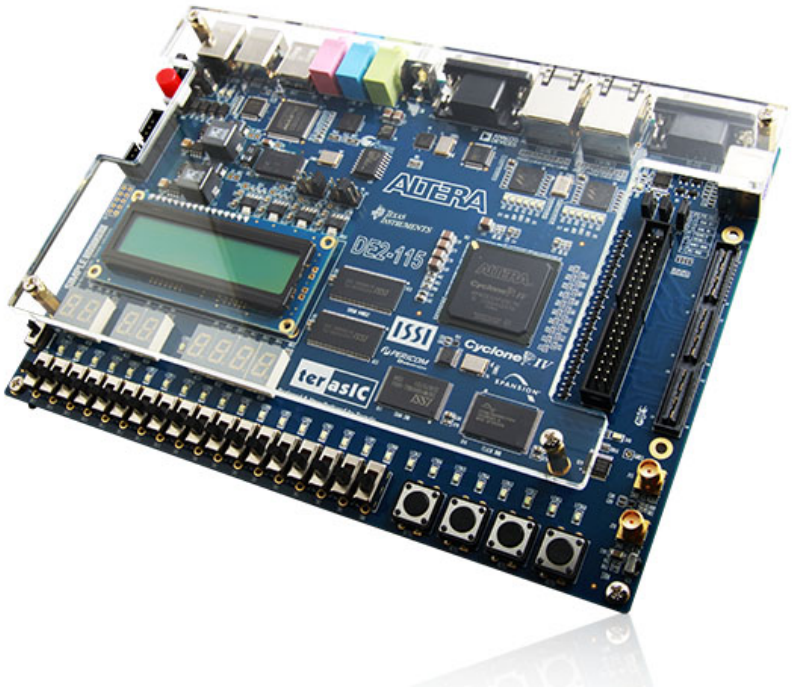


圖 2.1、DE2-115 教育開發平台[3]

表 2.1、Cyclone IV EP4CE115 FPGA 晶片資源

資源名稱	資源數量
邏輯元件	114480 LEs
內嵌記憶體	3888 Kbits
內嵌乘法器	266 個
PLL	4 個
User IO	528 Pin

表 2.2、DE2-115 外部記憶體資源

記憶體名稱	記憶體大小
SDRAM	128 MB
SRAM	2 MB
FLASH	8 MB
EEPROM	32 Kbits

2.1.2 硬體開發環境

本論文針對硬體開發而使用到的開發工具如表 2.3 所示，並在本小節說明各個開發工具的扮演的角色和用途。

表 2.3、硬體開發工具

工具名稱	簡易功能描述
Quartus II	RTL 電路合成、繞線、佈局、燒錄 FPGA
SignalTAP II	嵌入式邏輯分析儀
RTL Viewer	產生 RTL 電路方塊圖和電路間關係
Qsys	圖形化介面的 SOPC 設計工具
Icarus Verilog	開源的 RTL 模擬器與合成器
GTKWave	開源的邏輯分析儀模擬器

Quartus II 是 Intel FPGA 的 EDA 開發工具之一，其用途為將硬體描述語言編譯後合成出 RTL(Register-Transfer Level)電路，並針對該型號之 FPGA 晶片，進行繞線和佈局最後以 SOF(SRAM Object File)檔案(一種 Bitstream 檔案格式)來配置 FPGA；Quartus II 內部還有許多輔助工具如 SignalTAP II, RTL Viewer, Qsys，來幫助設計更複雜的系統電路以及時序分析。

SignalTap II 是一種嵌入式邏輯分析儀，他會在既有的硬體電路設計上，加入信號擷取電路，擷取後透過 JTAG 將資料傳回 Host PC 端，以此來觀測電路執行時的波形。

RTL Viewer 可以在 Quartus II 合成完電路後輸出 RTL 電路圖，除了能觀看電路拓樸關係，還能檢查電路間的連接是否正確。

Qsys 前身叫做 SOPC Builder，現在最新的版本又改稱為 Platform Designer，此工具是一款圖形化介面的 SOPC 設計工具，可以透過呼叫 IP(Semiconductor Intellectual Property)庫裡的元件來設計出自己的 SOPC，

亦可加入自己設計的 IP，透過點選連線的方式來決定電路的拓樸關係，省去以往人工佈線，容易出錯的問題，大大加快了嵌入式系統(Embedded System)的開發流程。

GTKWave 是一款開源的邏輯分析儀模擬器，可以繪出波形紀錄檔的波形，來驗證 RTL 的功能是否正確，其資料格式的型態齊全，可以在有號數、無號數、二進制、十六進制甚至是浮點數間進行格式轉換，且每條信號能有不同顏色的強調，對硬體開發者而言是一款很方便的除錯工具。

Icarus Verilog 是一款開源的 Verilog 硬體描述語言模擬器與合成器，撰寫 Testbench 對待測電路進行測試，執行完後輸出波形紀錄檔，可由 GTKWave 繪出波形，達到 Cycle Time 等級的精確除錯功能，其編譯速度和模擬執行速度表現都很優異，Icarus Verilog 搭配 GTKWave 是我目前最常拿來驗證 RTL 電路功能的最佳工具。

2.1.3 軟體開發環境

本論文選用開源的 C 語言編譯器工具鏈 RISC-V GNU Compiler Toolchain，這是一款支援 RISC-V 指令集的 C/C++的編譯器，裡面還包含了其他工具如組譯器(Assembler)、連結器(Linker)和二進制實用工具(Binary utilities)，對軟體開發者來說是不可或缺的好工具。

在 RISC-V 國際協會的 GitHub 網站上[4]，可以下載到 RISC-V GNU Compiler Toolchain，在網站上也有相關的安裝步驟說明。

2.2 系統架構

系統架構如圖 2.2 所示，整個系統架構中包含了 Picorv32 處理器、Avalon Bridge 橋接器、Memory 記憶體、Debug module 除錯模組以及其

它周邊 IO 控制器，連接到 Avalon 系統匯流排上整合為一個 SOPC 系統，將程式下載到記憶體後，搭配周邊 IO 就能進行一個初步的展示和驗證，以下將簡單介紹每個模組的功能。

1. Picorv32 處理器

   為系統中的處理器，負責程式的執行，運算和控制，是整個系統中的大腦。

2. Avalon Bridge 橋接器

   負責橋接 Picorv32 處理器與 Avalon 匯流排，在架構圖中稱此橋接器命名為 Avalon Bridge，在實作電路中取名為 Avalon Adapter。

3. Main Memory 主記憶體

   儲存程式碼和運算資料，可為 SDRAM、On-Chip Memory、SRAM 或是 Flash 等記憶體元件。

4. Debug module 除錯模組

   下載程式的工具，讓軟體開發者在 Host PC 端開發完程式後可經由序列傳輸，將程式下載到記憶體中，並回報除錯資訊。

5. Others Peripheral 其它周邊

   如 LED 燈、七段顯示器、按鈕開關、指撥開關、序列傳輸介面。

6. Avalon Switch Fabric

   將以上元件整合到系統匯流排中，若發生資源競爭，Qsys 工具將自動在其中做出仲裁邏輯。

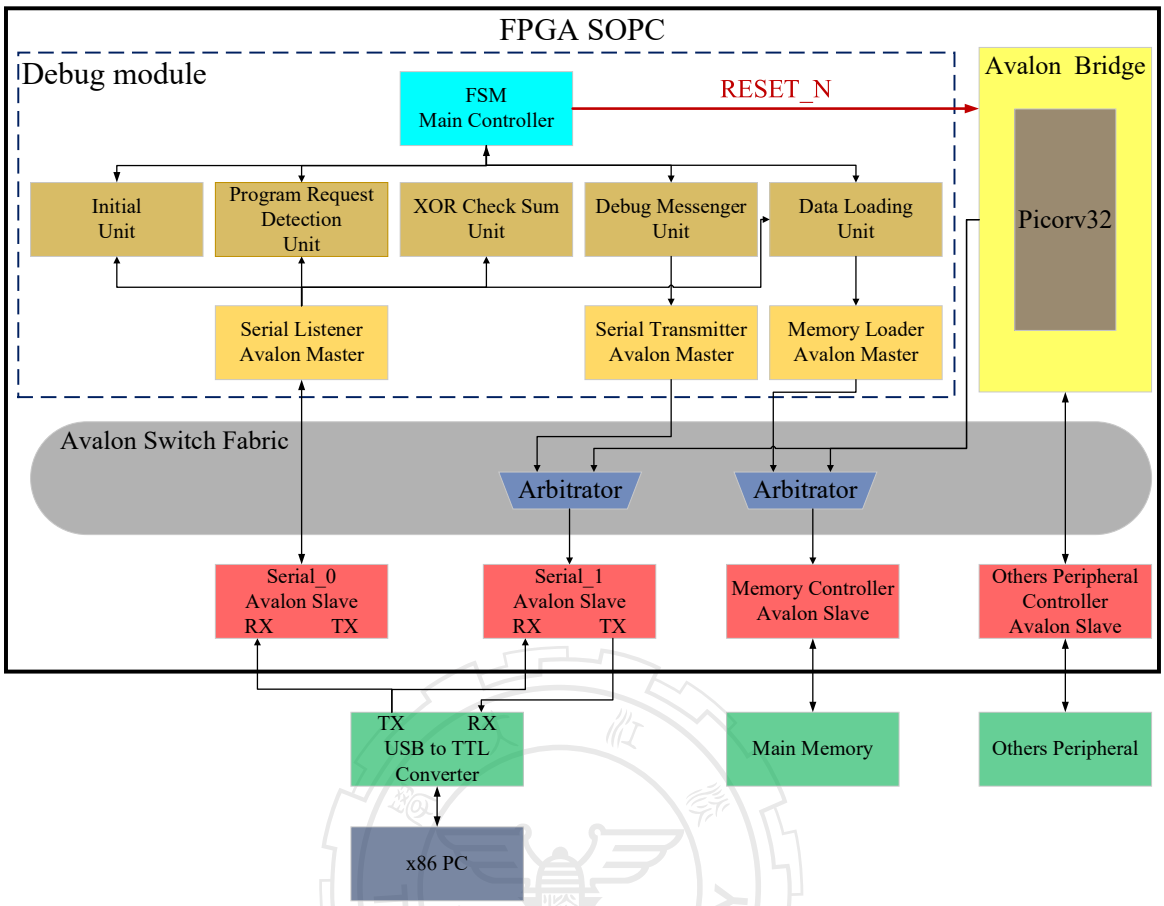

FPGA SOPC
Debug module
FSM
Main Controller
RESET_N
Avalon Bridge
Picorv32
Initial
Unit
Program Request
Detection
Unit
XOR Check Sum
Unit
Debug Messenger
Unit
Data Loading
Unit
Serial Listener
Avalon Master
Serial Transmitter
Avalon Master
Memory Loader
Avalon Master
Avalon Switch Fabric
Arbitrator
Arbitrator
Serial_0
Avalon Slave
RX TX
Serial_1
Avalon Slave
RX TX
Memory Controller
Avalon Slave
Others Peripheral
Controller
Avalon Slave
TX RX
USB to TTL
Converter
Main Memory
Others Peripheral
x86 PC

圖 2.2、系統架構

第三章 硬體設計

在本章節介紹 FPGA 端數位電路的設計方法，包含了橋接器設計、傳輸介面的時序分析、Picorv32 處理器的參數修改、Picorv32 處理器與 Avalon 匯流排該如何橋接，如何來建置 SOPC，最後設計用來幫助程式下載的 Debug module。

3.1 橋接器設計

為了讓 Picorv32 處理器的原生記憶體介面能夠與 Avalon 匯流排進行通信，為此設計兩者之間的橋接器，其橋接參考了 Wishbone 匯流排和 Avalon 匯流排的橋接方法[5]，匯流排信號大同小異，先觀察出相同功能的信號將其連接，不同功能的信號通常也僅需少許的組合邏輯即可完成連接，Wishbone 匯流排是一種開源的匯流排架構，其架構簡單，常被應用在嵌入式系統中主僕電路間的通訊架構中，訊號腳位和功能與 Avalon 匯流排相似。

Avalon 匯流排、Picorv32 處理器和橋接器的架構如圖 3.1 所示，橋接器做為兩者間的溝通橋梁，將 Picorv32 處理器的原生記憶體介面轉換為兩個 Avalon Memory-Mapped Master 介面，其一是 Instruction Master，負責指令讀取的主介面，其二則是 Data Master，負責資料讀寫的主介面，設計成與 Nios II 處理器一樣分別有 Instruction Master 和 Data Master 介面。

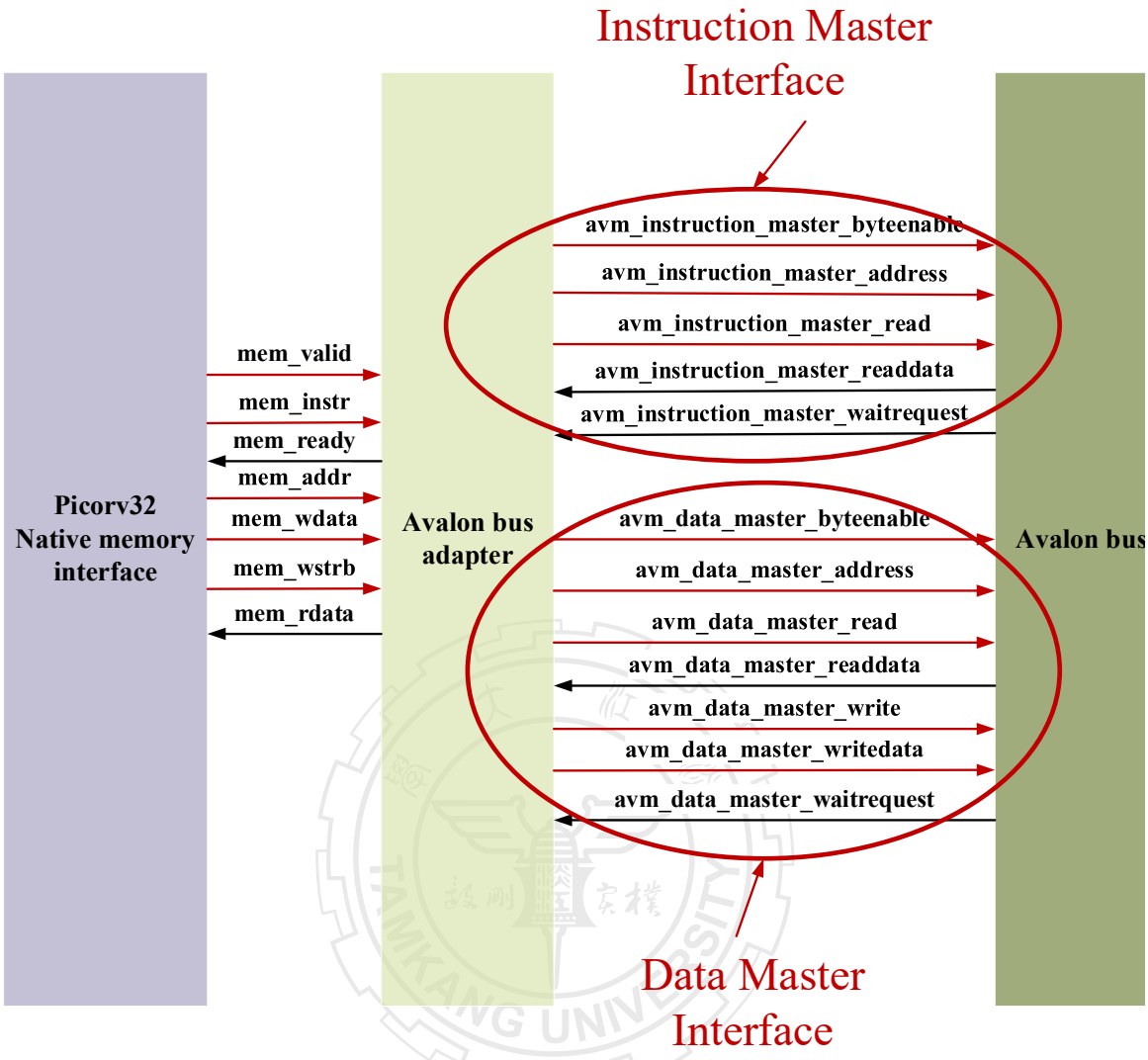


圖 3.1、橋接器架構

Picorv32 處理器的原生記憶體信號說明如表 3.1 所示，橋接器的輸入輸出以及位元寬度定義如圖 3.2 所示。

表 3.1、Picorv32 原生記憶體介面信號說明

信號名稱	功能說明
mem_addr	讀寫地址
mem_valid	當欲進行讀寫傳輸時會拉起，相當於 read 和 write 命令
mem_instr	當執行指令讀取操作時為 1
mem_ready	為 1 時，代表成功寫入或是可以讀取資料
mem_wdata	寫入資料
mem_wstrb	如同 Avalon-MM byteenable，但當讀操作時恆為 4'b0000
mem_rdata	讀取資料

```
module picorv32_avalon_adapter
(
    //native memory interface
    input         mem_valid,   //begin memory read or write
    input         mem_instr,   //instruction
    output        mem_ready,   //waitrequest
    input  [31:0] mem_addr,    //address
    input  [31:0] mem_wdata,   //write data
    input  [3:0]  mem_wstrb,   //write byteenable
    output [31:0] mem_rdata,   //read data

    //avalon-MM instruction master interface
    output [3:0]  avm_instruction_master_byteenable,
    output [31:0] avm_instruction_master_address,
    output        avm_instruction_master_read,
    input  [31:0] avm_instruction_master_readdata,
    input         avm_instruction_master_waitrequest,
    //avalon-MM data master interface
    output [3:0]  avm_data_master_byteenable,
    output [31:0] avm_data_master_address,
    output        avm_data_master_read,
    input  [31:0] avm_data_master_readdata,
    output        avm_data_master_write,
    output [31:0] avm_data_master_writedata,
    input         avm_data_master_waitrequest
);
```

圖 3.2、橋接器輸入輸出和位元寬度定義

先定義好通訊協定規格並畫出時序圖，再設計其內部橋接邏輯，使其符合通訊協定要求，同時要符合 Picorv32 處理器的原生記憶體介面和 Avalon Memory-Mapped 介面的通訊協定，以下將針對各個時序圖，做詳細說明。

Avalon Memory-Mapped 通訊協定如圖 3.3 所示，其信號說明如表 3.2 所示。

Avalon Master 進行讀取時，分為以下幾個步驟進行：

步驟一：先準備好 address 和 byteenable，再拉起 read 訊號，同時 Slave 端的 waitrequest 也會跟著拉起。

步驟二：當 Slave 準備好資料後，降下 waitrequest，此時 Master 才可以在下一個時脈正緣觸發時讀取 readdata 資料，並降下 read 命令。

Avalon Master 進行寫入時，分為以下幾個步驟進行：

步驟一：先準備好 address、byteenable 和欲寫入的資料 writedata，再拉起 write 訊號，同時 Slave 端的 waitrequest 也會跟著拉起。

步驟二：等待 waitrequest 降下後，代表資料成功寫入該記憶體位置，在下個時脈正緣觸發時即可降下 write 命令。

此時序圖需要注意的地方是，真實 waitrequest 信號在非傳輸時是恆為 1 的，此圖只是想強調傳輸從何時開始，並非真實情況，Intel 官方建議，在 IDLE 時 waitrequest 常設為 1。

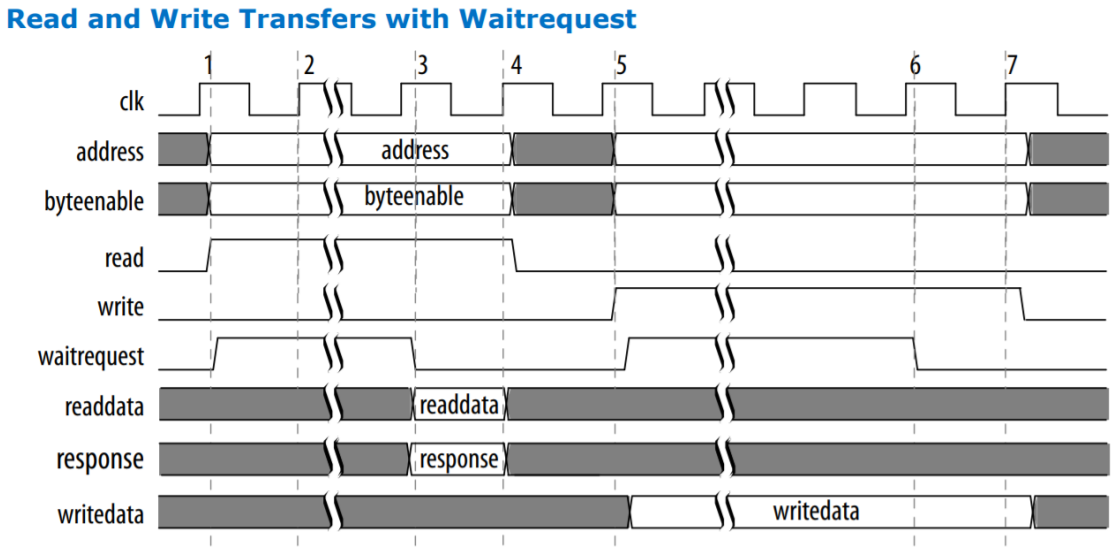


圖 3.3、Avalon Memory-Mapped 介面典型讀寫時序[6]

表 3.2、Avalon Memory-Mapped 介面信號說明

信號名稱	功能說明
address	讀寫地址
byteenable	位元組致能，可以決定 4 Byte 的資料，哪一個 Byte 有效
read	Master 讀命令
write	Master 寫命令
waitrequest	等待請求，當 Slave 資料尚未準備好時此信號會被拉起

Picorv32 的原生記憶體介面信號說明如表 3.1 所示，指令讀取時序如圖 3.4 所示，指令讀取動作分成以下三個步驟說明：

步驟一：準備好讀取的記憶體位置 mem_addr，再拉起 mem_valid 代表要進行記憶體傳輸，mem_wstrb 設為 4'b0000，代表進行讀操作，mem_instr 設為 1 代表是讀取指令而非一般資料讀寫。

步驟二：在 mem_ready 升起時表示指令資料已經準備好了，並在下一個時脈正緣觸發時降下 mem_valid 代表結束通訊。

步驟三：mem_instr 在結束通訊後維持一個時脈週期後降下，經 SignalTap II 波形驗證無誤，如圖 3.5 所示。

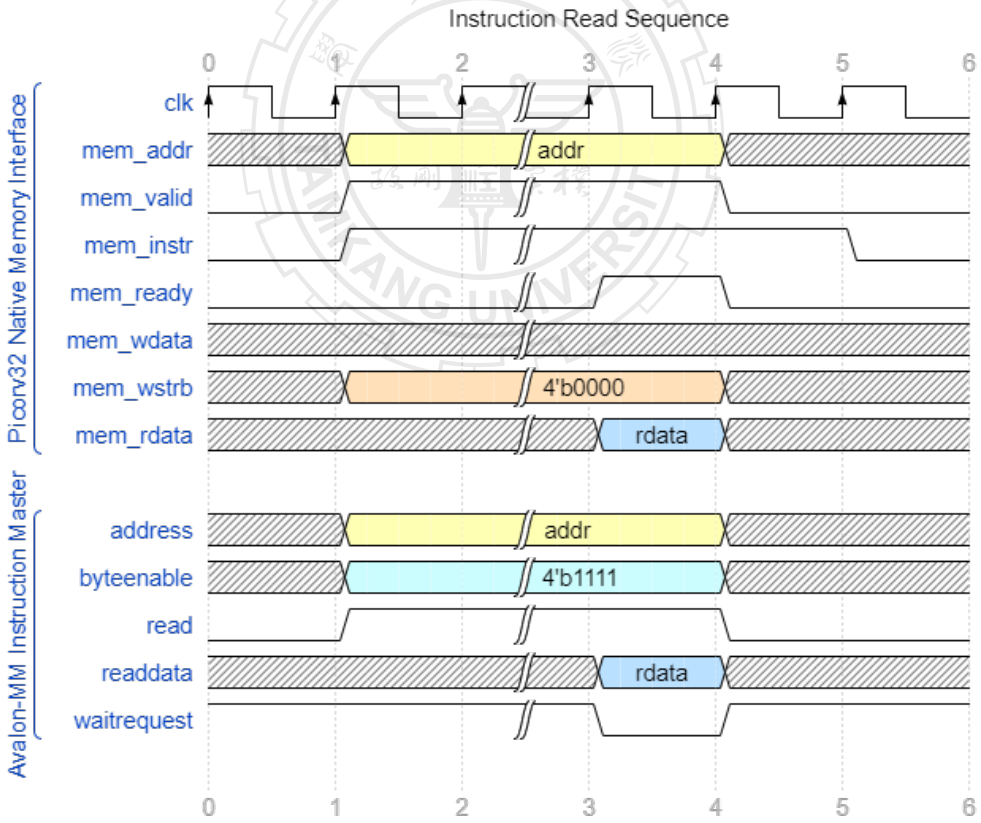


圖 3.4、指令讀取時序

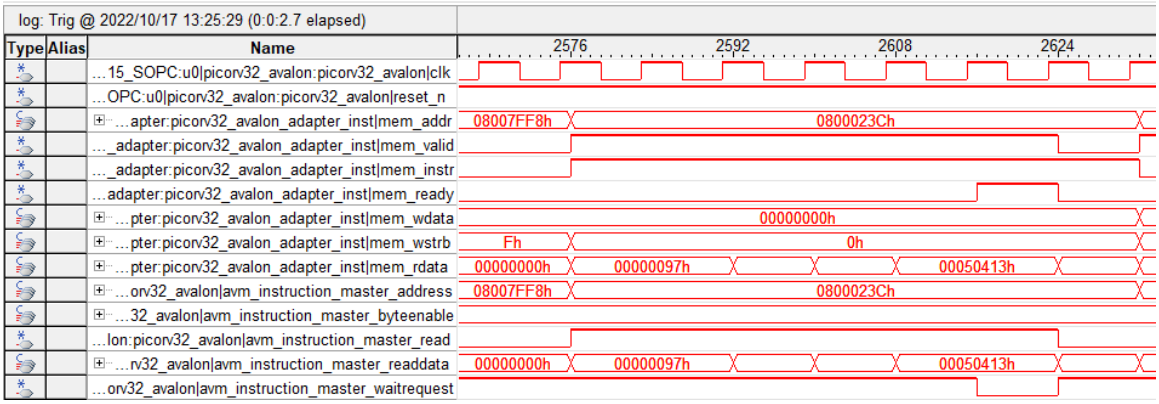


圖 3.5、SignalTap II 指令讀取時序

資料讀取時序如圖 3.6 前半段所示，資料讀取動作分成以下兩個步驟說明：

步驟一：準備好讀取的記憶體位置 mem_addr，再拉起 mem_valid 代表要進行記憶體傳輸，mem_wstrb 設為 4'b0000 代表進行讀操作，mem_instr 設為 0 代表是讀取資料而不是指令。

步驟二：當 mem_ready 升起時代表資料已經準備好了，在下一個時脈正緣觸發時收走 mem_rdata，並降下 mem_valid 代表結束通訊，經 SignalTap II 波形驗證無誤，如圖 3.7 所示。

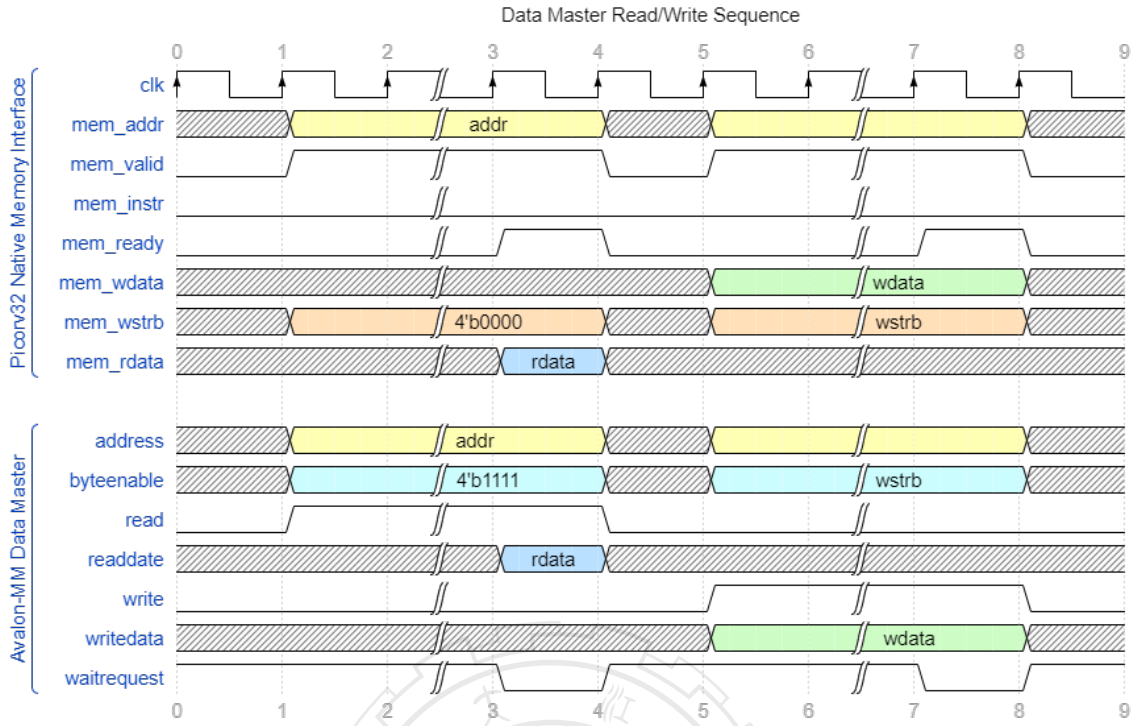


圖 3.6、資料讀寫時序

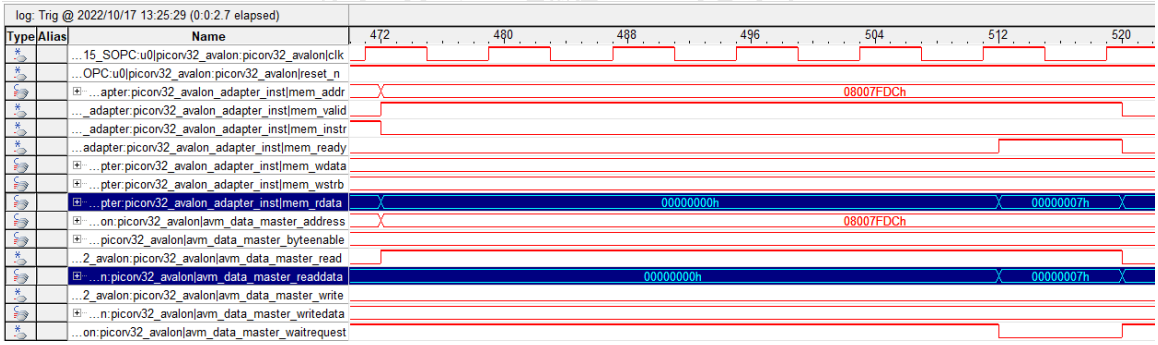


圖 3.7、SignalTap II 資料讀取時序

資料寫入時序如圖 3.6 後半段所示，資料寫入動作分為以下兩個步驟說明：

步驟一：準備寫入的記憶體位置 mem_addr，再拉起 mem_valid 代表要進行記憶體傳輸，mem_wstrb 在寫操作時可以配置成 4’b0000、4’b1111、

4’b1100、4’b0011、4’b1000、4’b0100、4’b0010 和 4’b0001 八種組態的其中一種，代表要讓 4 Bytes 資料中的哪幾個 Byte 有效，mem_instr 設為 0 代表與指令讀取無關。

步驟二：當 mem_ready 升起時，代表資料成功寫入記憶體，並在下個時脈正緣觸發時降下 mem_valid 代表結束通訊，經 SignalTap II 波形驗證無誤，如圖 3.8 所示。

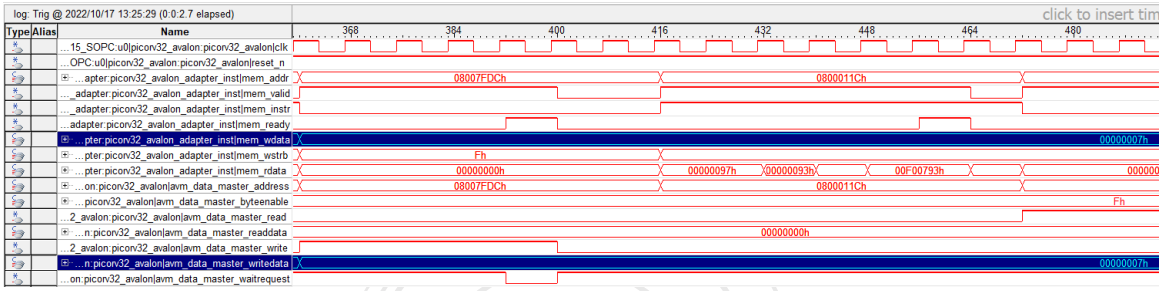


圖 3.8、SignalTap II 資料寫入時序

觀察以上 Picorv32 處理器的讀寫操作配合 Avalon Memory-Mapped 時序圖我們可以輕鬆的決定橋接器的內部邏輯，主要可以分成三塊輸出邏輯，分別是 Instruction Master Output Logic、Data Master Output Logic 和 Picorv32 Output Loigc 如圖 3.9 所示。

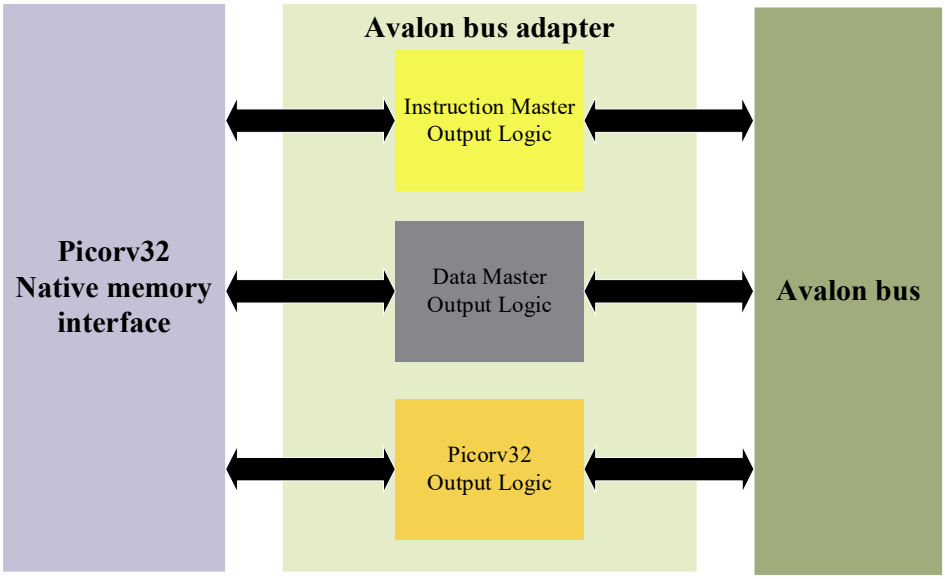


圖 3.9、橋接器內部三塊輸出邏輯

其 Verilog 程式碼如圖 3.10 所示，僅需簡單的組合邏輯即可完成，下文將搭配 RTL 電路圖來輔助闡述橋接思路。

```
28      //avalon instruction master
29      assign avm_instruction_master_byteenable = 4'b1111;
30      assign avm_instruction_master_address = mem_addr;
31      assign avm_instruction_master_read = mem_valid && mem_instr;
32      //avalon data master
33      assign avm_data_master_byteenable = (avm_data_master_write) ? mem_wstrb : 4'b1111;
34      assign avm_data_master_address = mem_addr;
35      assign avm_data_master_read  = mem_valid && (mem_wstrb == 4'b0000) && !mem_instr;
36      assign avm_data_master_write = mem_valid && (mem_wstrb != 4'b0000) && !mem_instr;
37      assign avm_data_master_writedata = mem_wdata;
38      //native memory
39      assign mem_ready = (mem_instr) ? !avm_instruction_master_waitrequest : !avm_data_master_waitrequest
40      assign mem_rdata = (mem_instr) ?  avm_instruction_master_readdata : avm_data_master_readdata;
41
```

圖 3.10、橋接器內部邏輯 Verilog Code

Avalon Instruction Master 端的輸出邏輯較為簡單，其 RTL 電路如圖 3.11 所示，每次讀取指令的記憶體大小固定為 4 Bytes，因此 master_byteenable 設為定值 4’b1111，master_address 則是與 Picorv32 處理器原生記憶體介面的 mem_addr 對接，不需要做多餘的判斷邏輯，其原因

是 Instruction Master 會依照 master_read 信號來判斷現在要不要進行指令讀取，只要不是在 master_read 的狀況下 master_address 的數值就會被忽略，最後一個 master_read 訊號的輸出邏輯是 mem_valid & mem_instr，mem_valid 信號代表進行記憶體傳輸而 mem_instr 決定是否為 Instruction 的傳輸而非 Data 的傳輸，當 mem_valid 和 mem_instr 信號都為 1 時就能知道是要讀取 Instruction。

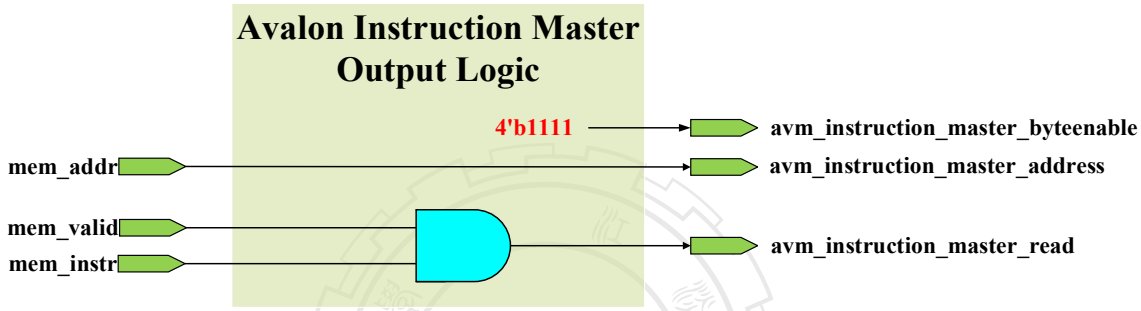


圖 3.11、Avalon Instruction Master Output Logic RTL View

Avalon Data Master 端的輸出邏輯較為複雜一些，其 RTL 電路如圖 3.12 所示，master_address 和 master_writedata 可直接與 mem_addr 以及 mem_wdata 直接連接，在沒有進行傳輸時這兩個數值將會直接被忽視，因此不需要做多餘的判斷邏輯；當 master_write 為 1 時，也就是 master 進行寫入時，master_byteenable 應當等於 mem_wstrb，來決定哪幾個 Byte 應該被寫入，其餘時 master_byteenable 常設為 4'b1111；決定 master_read 是否要進行讀取，取決於三個輸入，分別是 mem_wstrb, mem_valid 和 mem_instr，可以依照 mem_wstrb 是否等於 4'b0000 來判斷是否進行讀取操作而非寫入操作，因為寫入操作時是不允許 mem_wstrb 為 4'b0000 的，而 mem_valid 可以判斷現在是否進行記憶體傳輸，最後一個 mem_instr 用

來判斷是要讀取 Instruction 還是 Data，同時符合 mem_wstrb 等於 4'b0000, mem_valid 等於 1 以及 mem_instr 等於 0 即可判斷出現在 Data Master 是否要進行讀取，而判斷 Data Master 是否要進行寫入跟以上的邏輯一樣，差別僅在於 mem_wstrb 的比較，非零值時代表寫入。

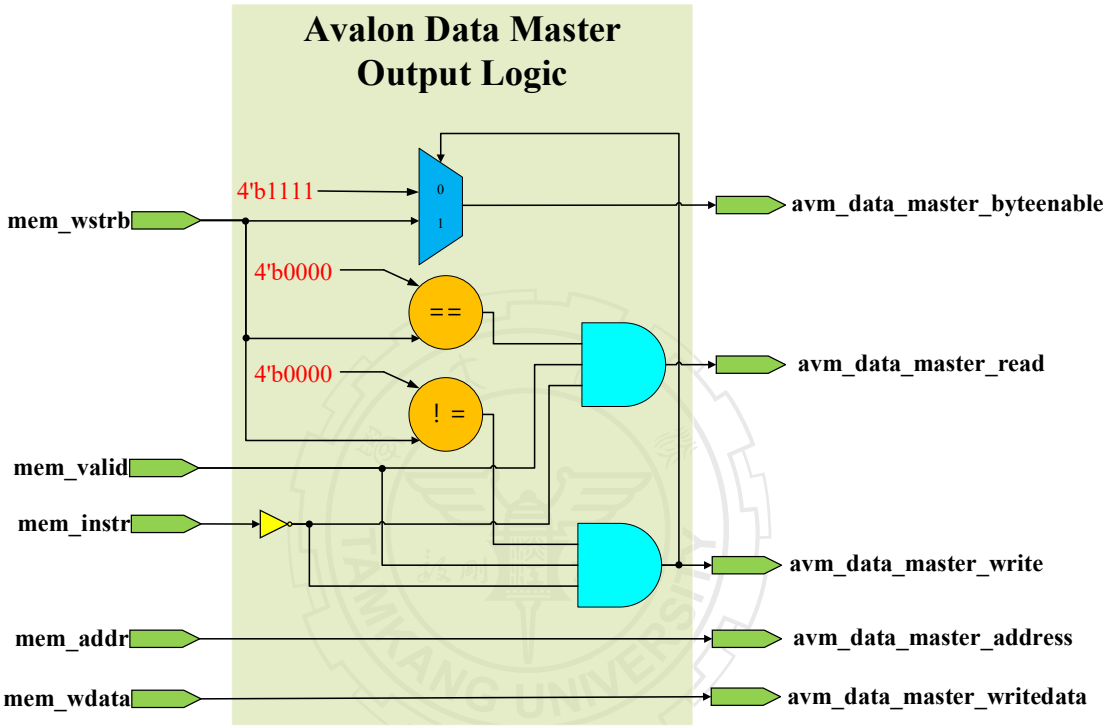


圖 3.12、Avalon Data Master Output Logic RTL View

Picorv32 處理器的原生記憶體介面端只有兩個輸出，其 RTL 電路如圖 3.13 所示，mem_ready 和 mem_rdata，mem_ready 代表記憶體成功寫入或是現在可以讀取資料，mem_rdata 代表讀取的數值，只需要用 mem_instr 來區分是讀取 Instruction 還是 Data，需要注意的地方是 master_waitrequest 是 Active Low，剛好與 mem_ready 相反，需要做反向。

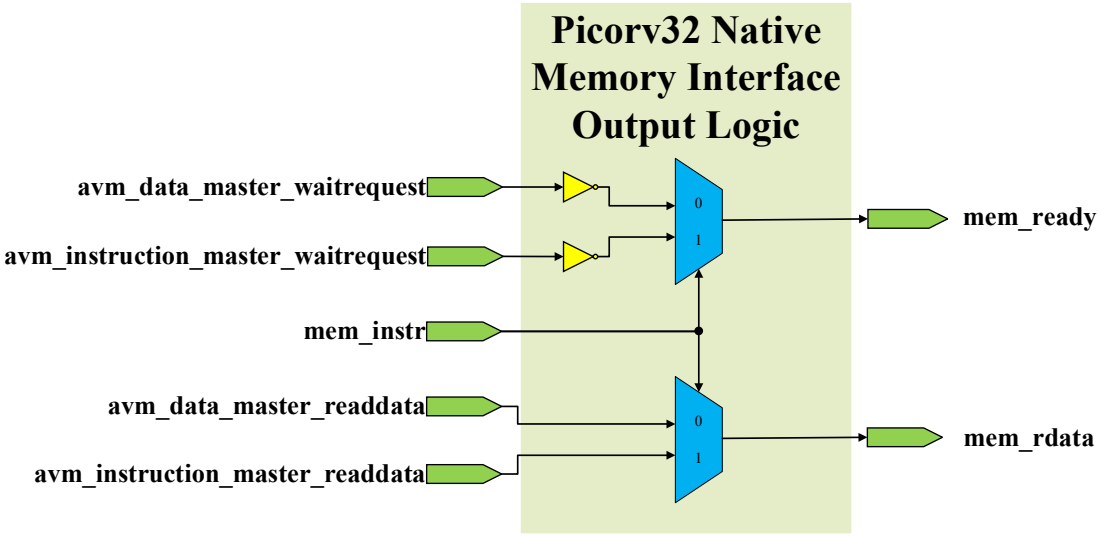


圖 3.13、Picorv32 Native Memory Interface Output Logic RTL View

3.2 Picorv32 處理器介紹和參數修改

Picorv32 處理器是一款基於 RISC-V 指令集架構的開源處理器核心，授權方式採用 ISC(Internet System Consortium)，主要是由 Claire Xenia Wolf 與其他自由軟體工作者，以 Verilog 硬體描述語言撰寫的軟核心處理器，其特點就是資源使用量非常小，適合應用在低功耗的嵌入式系統上。

在挑選 RISC-V 處理器時，之所以選擇 Picorv32 處理器作為本論文的實驗對象，是因為它不僅邏輯資源使用量小，可以輕易的在 FPGA 上實現，而且是以 Verilog 硬體描述語言設計，對於熟悉 Verilog 的我來說較好上手，現今的硬體開發已不限於傳統的 Verilog 和 VHDL 兩種硬體描述語言，更多的是以高階語言來轉譯成低階的硬體描述語言，又或者直接以高階語言的形式合成電路，如 Sifive 公司的開源處理器 rocket[7]和 freedom[8]處理器核心就是以 Chisel 一種基於 Scala 的高階程式語言來設計的硬體。

Picorv32 提供了許多的參數化的設計，讓使用者可以更靈活地去配置和修改，可以將其配置成 RV32E、RV32I、RV32IC、RV32IM 與 RV32IMC 指令集架構，也能針對性能做出調整，其後綴字母為基礎指令集搭配擴增指令集。

表 3.3 所示，其詳細指令集規格參閱 RISC-V 官方規格書[9]，此小節僅列出需要修改的參數，配置成最簡單的 RV32I 指令集，參數說明如表 3.4 所示，更詳細的參數說明請參閱 Picorv32 官方的 GitHub[10]。

表 3.3、指令集說明

後綴字母	指令集說明	指令集種類
E	嵌入式整數指令集，僅有 16 個 32bit 暫存器	基礎指令集
I	整數指令集，含有 32 個 32bit 暫存器	基礎指令集
M	整數乘除法指令集	擴增指令集
C	壓縮指令集	擴增指令集

表 3.4、參數說明

參數名稱	簡易說明
COMPRESSED_ISA	支援壓縮指令集
ENABLE_REG_16_31	選用 16 或 32 個 32 位元的暫存器
ENABLE_REGS_DUALPORT	Register file 雙讀取 port，增加讀取速度
ENABLE_PCPI	致能協處理器，支援乘除法運算
ENABLE_IRQ	致能中斷功能
PROFADDR_RESET	程式起始位置
STACKADDR	Stack pointer 起始位置

以下將說明參數的設定數值以及原因。

COMPRESSED_ISA 設定為 0，因為我們要配置的是 RV32I，32 位元的的整數指令集所以必須關閉此項設定。

ENABLE_REG_16_31 設定為 1，RV32I 指令集包含了 32 個 32 位元的暫存器，需致能第 16~31 個暫存器。

ENABLE_REGS_DUALPORT 設定為 1，增加 register file 的讀取 port，可以增加讀取效率。

ENABLE_PCPI 設定為 0，並未啟用協處理器，乘除法的運算是用整數指令集的運算來模擬的，並無用到硬體的乘除法器。

ENABLE_IRQ 設定為 0，並未加入中斷的實驗，因此並未開啟。

PROGADDR_RESET 與 STACKADDR，需要看記憶體被擺放在處理器定址空間的何處，以及軟體程式下載到記憶體後的 Memory layout 才能決定，如所圖 3.14 所示。

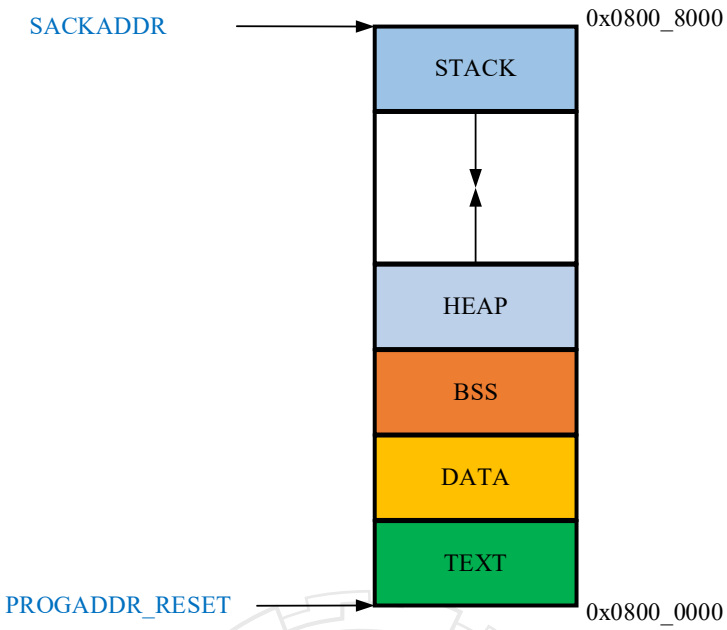


圖 3.14、Memory layout

3.3 SOPC 設計

當 Picorv32 處理器橋接完畢後即可開始搭建 SOPC，有了處理器、記憶體、和周邊 IO 控制器，系統就能夠運行起來，本小節將講述如何透過 Qsys 工具將橋接後的 Picorv32 處理器掛上 Avalon 系統匯流排，與其他的周邊形成一個晶片系統，最後讓 Qsys 幫我們產生整個系統的 RTL Code，方便後續到 Quartus II 上進行合成繞線和布局，做出屬於自己的 SOPC，其步驟如下。

1. 打包 Picorv32 處理器和橋接器。

   Picorv32 處理器和橋接器是兩個不同的電路，必須整合為一個 Avalon Master 電路 IP 如圖 3.15 所示，方便 Qsys 工具識別。

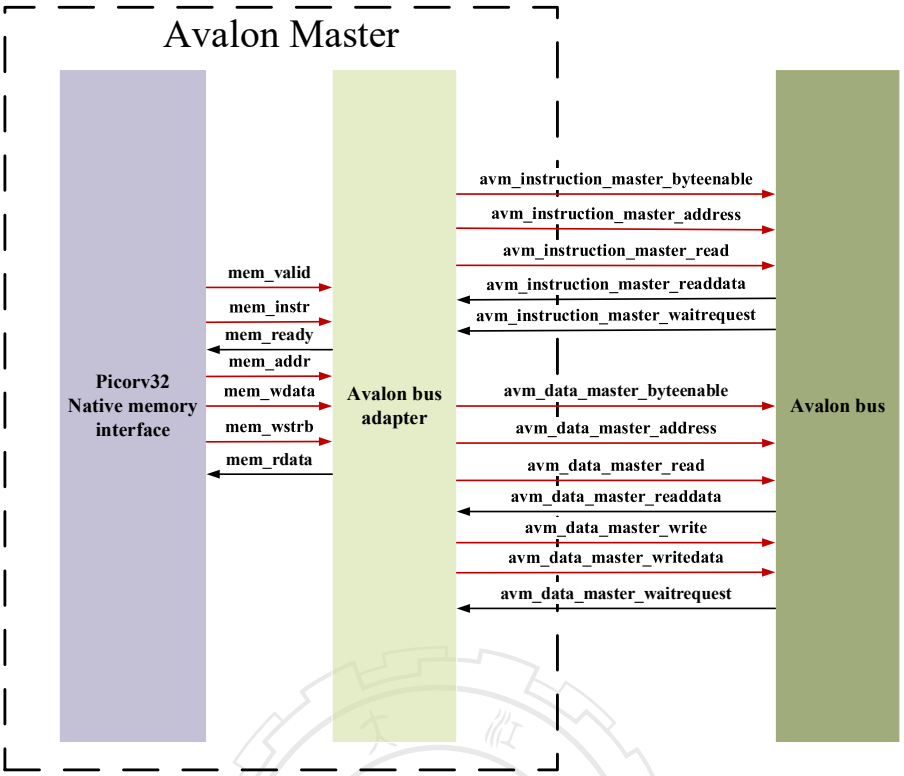


圖 3.15、打包 Picorv32 和橋接器

2. 驗證分析電路

在 Qsys 裡面打開 Component Editor 後加入表 3.5 中三個檔案，設定 picorv32_avalon.v 檔案為 Top-Level File，並對些檔案做分析確認無語法錯誤，為可合成之數位電路，如圖 3.16 所示。

表 3.5、各檔案對應電路

檔案名稱	對應電路
picorv32.v	Picorv32 核心
picorv32_avalon_adapter.v	橋接器
picorv32_avalon.v	Avalon Master

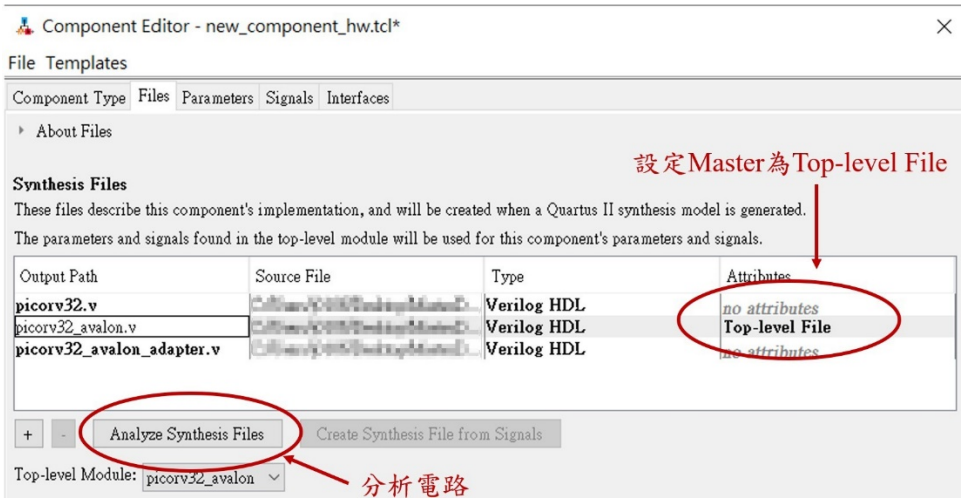


圖 3.16、分析檔案

3. 確認信號和介面

若腳位名稱有符合 Avalon Bus 規格書上的命名規則[11]，在此步驟 Qsys 將自動識別 Signal Type，如所圖 3.17 示。

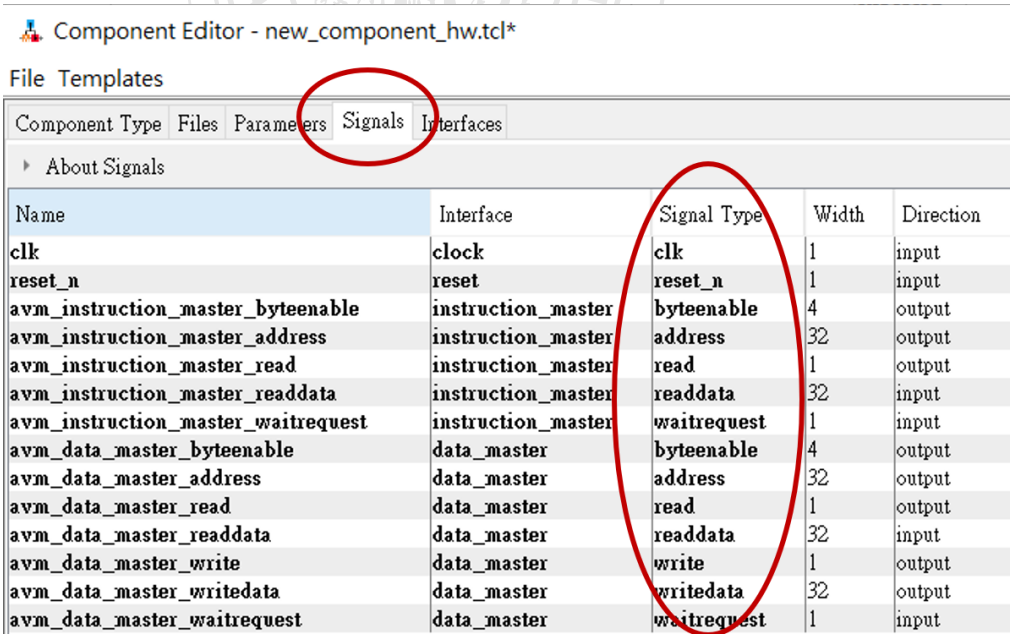


圖 3.17、確認信號

檢查完信號後要確認介面，會發現錯誤回報視窗有兩條錯誤訊息 Error:instruction_master:Interface must have an associated reset 和 Error:data_master:Interface must have an associated reset 意思是 Instruction Master 和 Data Master 介面沒有接上系統的 reset，按照圖 3.18 和圖 3.19 所示將其接上，完成這步就算掛上 Avalon 系統匯流排了。

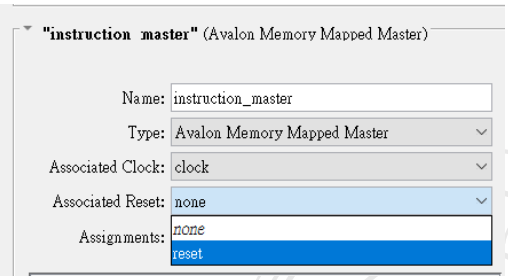


圖 3.18、Instruction Master Associated Reset

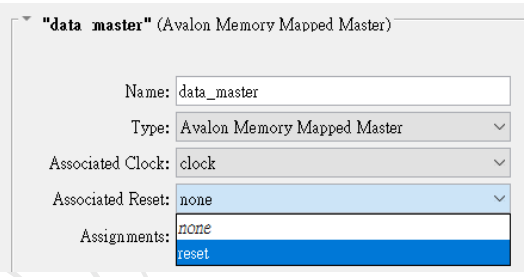


圖 3.19、Data Master Associated Reset

4. 新增記憶體元件和周邊 IO 控制器

透過點選 Qsys IP Library 來新增我們要的元件，如處理器、記憶體和周邊 IO 控制器如圖 3.20 所示，確定好系統裡的元件後就可以開始連接電路，picorv32_avalon 的 Instruction Master 和 Data Master 需連接到系統中的主記憶體 On-Chip Memory，為了方便在此章節中命名為 SRAM，但在 DE2-115 開發平台上亦有外部的 SRAM 晶片，在此特別註明，兩者是不一樣的。

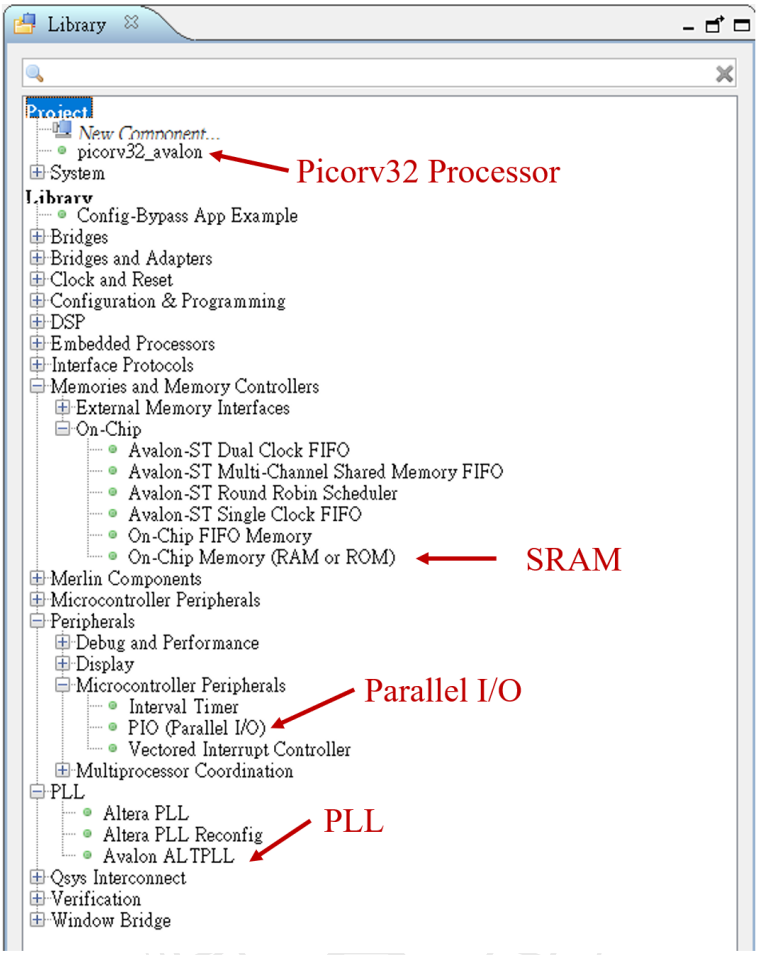


圖 3.20、Qsys IP Library

Data Master 還需與其他周邊元件如指撥開關、按鈕開關、LED 燈、七段顯示器等的 PIO 控制器連接，以記憶體映射的方式控制這些元件，其元件間拓樸關係如圖 3.21 所示。

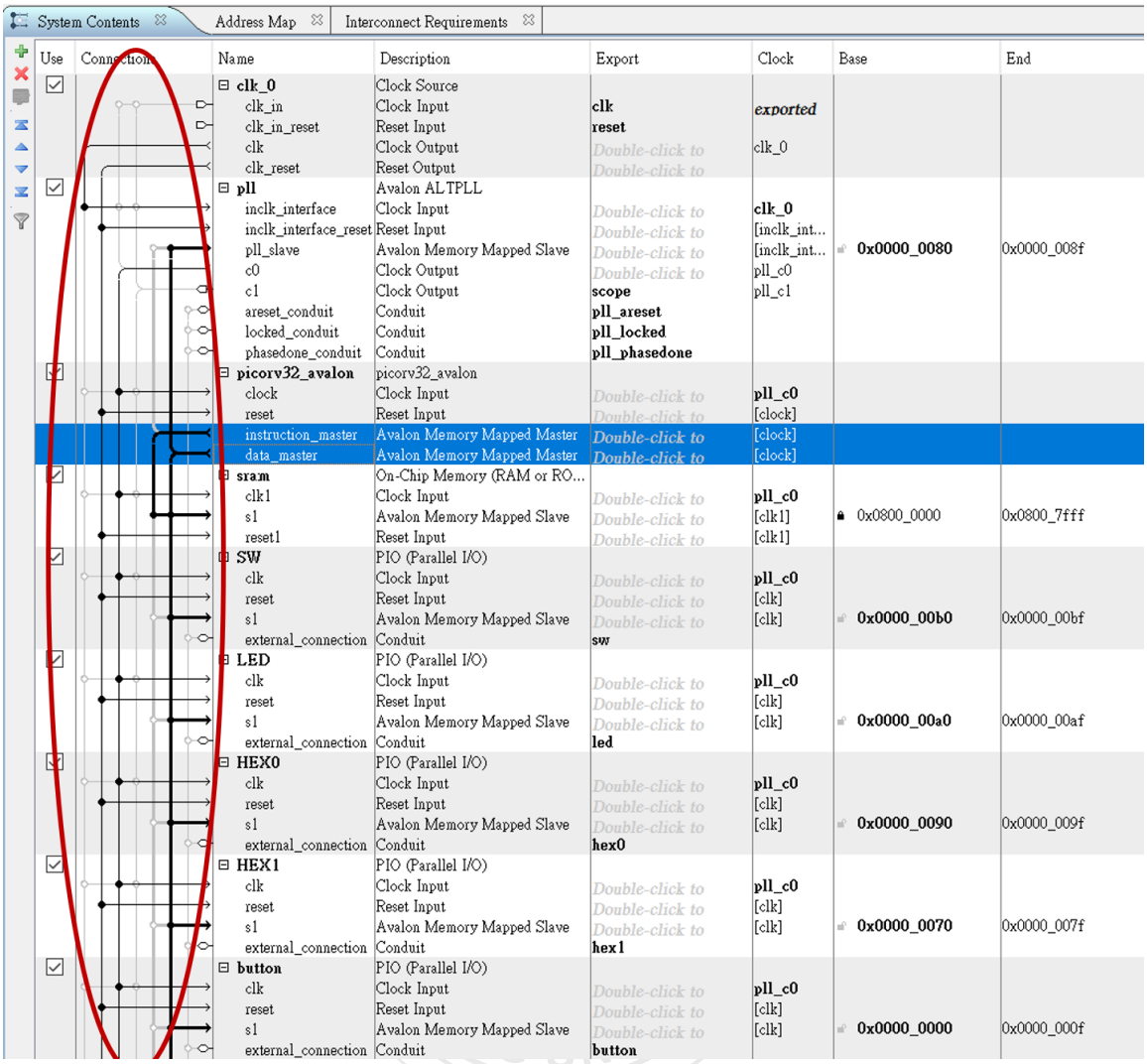


圖 3.21、SOPC interconnection

5. 分配 Slave 元件的記憶體基底地址

如圖 3.22 所示，在上方 Tool bar 按下 Assign Base Address 可為周邊 Slave 分配記憶體基底地址。

其他周邊的基底地址可由系統隨機分配，唯獨 On-Chip Memory 的基底地址必須鎖定，On-Chip Memory 做為主記憶體使用，程式是儲存在裡面的，以後若要增加周邊，進行基底地址重新分配時，如果動到了 On-Chip Memory 的基底地址，程式的 Start

Address 和處理器裡的 Program Counter 初始值都必須要修改，程式才得以順利執行，固定記憶體的基底地址，可以更有效的開發程式。

最後按下 Generate 就可以產生 SOPC 系統的 RTL Code。

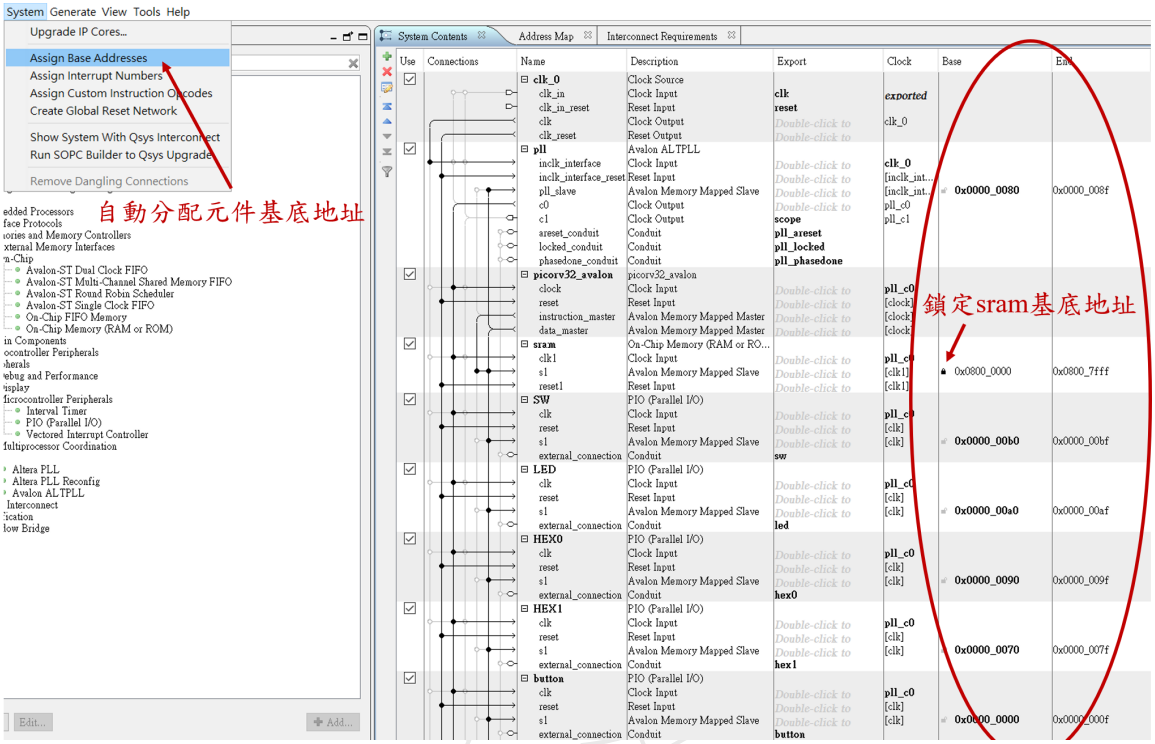


圖 3.22、分配記憶體映射空間

3.4 **Debug module 設計**

在系統架構中有提到了圖 2.2 中 Debug module 的用途為便利程式開發者，在 Host PC 端開發完 C 語言程式後，可以透過序列傳輸來下載程式到記憶體中，取代以 Hex File 初始化記憶體的方法，相當便利，以 Hex File 初始化記憶體的方式，必須將 Hex File 和硬體電路一起打包成 SOF 檔案來配置 FPGA，每次更改程式就必須重新合成硬體重新配置 FPGA，

非常費時。

在 Nios II 處理器中也能看見同樣功能的 JTAG Debug module 設計，但是由於我們必須要替換掉 Nios II 處理器，因此無法使用它自帶的 JTAG Debug module，為此著手設計自己的 Debug module。

Debug module 部分設計參考了 Reindeer 開源處理器的設計[12]，它設計了一個基於硬體的 Bootloader，可以透過序列傳輸下載程式到目標系統的記憶體中，不僅提供了硬體架構圖，並且提出了一個 Hold-and-Load 的機制，按下 reset 重置後處理器進入 hold 狀態等待程式上傳，程式上傳完畢後發送 start 脈衝信號來啟動處理器，本論文吸收其概念，實作出自己的 Debug module，包含有限狀態機(Finite-State Machine, FSM)和五個控制單元，搭配 Intel 免費授權的序列傳輸控制器 IP 以及記憶體控制器 IP，一起掛上 Avalon 系統匯流排，完成整個 Debug module 的設計，以下將介紹五個控制單元的工作和 FSM 的狀態圖。

1. Program Request Detection Unit

   在燒錄請求偵測單元中，其主要架構為一個序列檢測器(Sequential Detector)電路，當檢測到燒錄密碼時，就會開始啟動燒錄程序。

2. Initial Unit

   在初始化單元中，其主要架構為移位暫存器(Shift Register)電路，當燒錄程序剛啟動時，緊接著接收 Base address 和 Length 資訊，Base address 為程式下載到記憶體的起始地址，Length 為下載程式的長度，單位為 Byte。

3. XOR Check Sum Unit

   對傳進來的資料做 XOR 校驗檢查。

4. Debug Messenger Unit

透過序列傳輸傳遞除錯訊息給程式開發者。

5. Data Loading Unit

將序列傳輸收到的資料搬移到記憶體中。

6. FSM 有限狀態機

如圖 3.23 所示，當 Program Request Detection Unit 檢測到需要下載程式後狀態機從 Detect 狀態跳往 Init 狀態，並且重置 Picorv32 處理器，在 Init 狀態下 Initial Unit 會接收下載程式的起始地址和資料長度，當初始化完畢後會進行 XOR 的校驗碼計算，在 Report 狀態時命令 Debug messenger Unit 將除錯訊息透過序列傳輸給程式開發者，接著跳往 Loading 狀態將的序列傳輸收到的資料流(Data Stream)搬移到記憶體中，並設定計數器，紀錄搬移了多少筆資料，當計數器數值等於資料長度時，即代表下載完畢，之後再進行一次 XOR 校驗碼檢查，確保記憶體資料無誤，最後回到一開始的 Detect 狀態，放開重置訊號，讓處理器執行下載的程式。

在硬體上並未做過多的檢查和流程控制，可由軟體端收到 Report 資訊後重新上傳程式，或是設定計時器，當上傳過程卡住時可重新上傳程式。

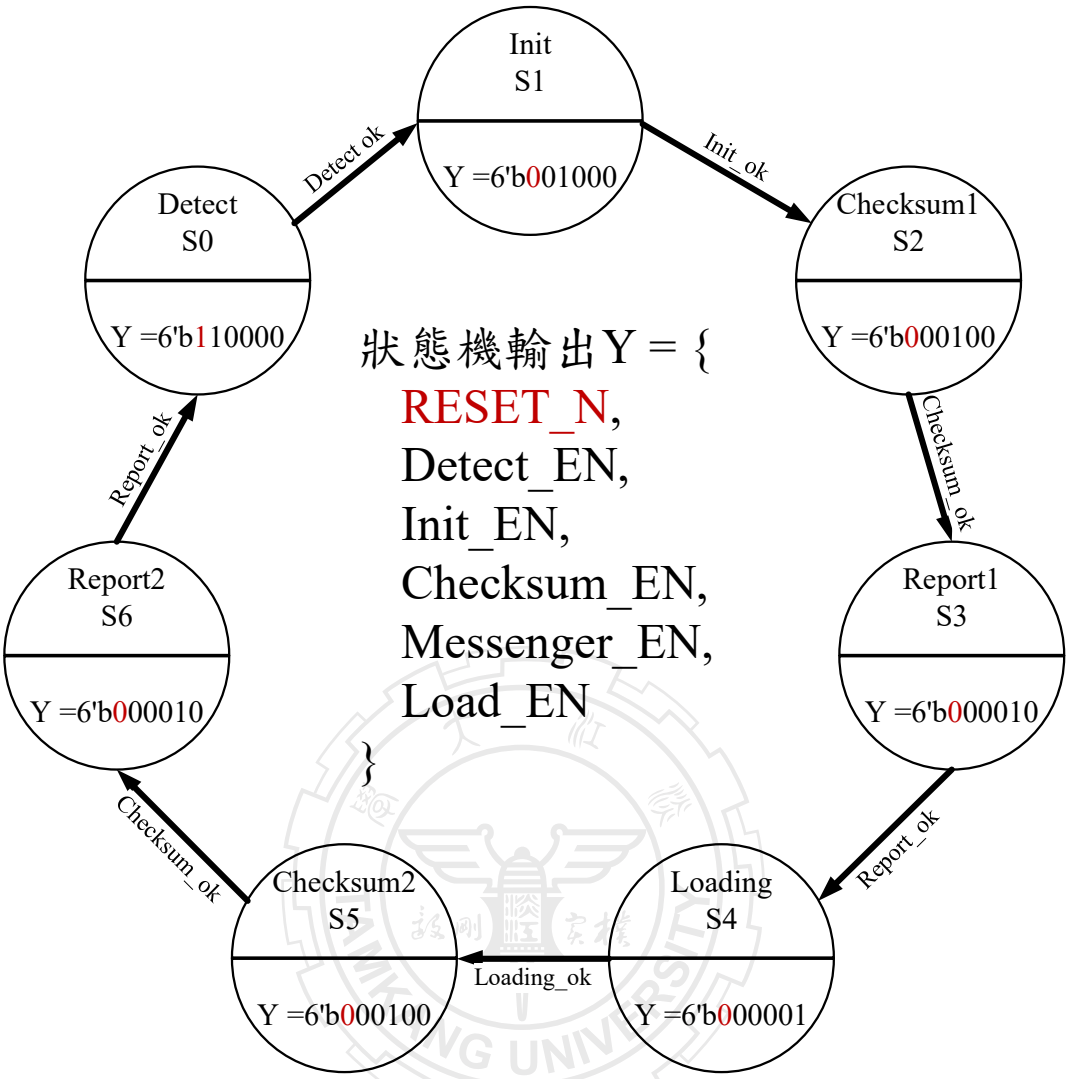

Init
S1
Y =6'b001000
Detect ok
Init_ok
Detect
S0
Y =6'b110000
Checksum1
S2
Y =6'b000100
狀態機輸出Y = {
RESET_N,
Detect_EN,
Init_EN,
Checksum_EN,
Messenger_EN,
Load_EN
}
Report_ok
Checksum_ok
Report2
S6
Y =6'b000010
Report1
S3
Y =6'b000010
Checksum_ok
Report_ok
Checksum2
S5
Y =6'b000100
Loading
S4
Y =6'b000001
Loading_ok

圖 3.23、FSM 狀態圖

第四章 軟體編譯和燒錄流程

本章節介紹了如何編譯 C 語言程式，交代其編譯流程，以及如何修改連結器腳本，並使用編譯器裡的工具對執行檔進行格式轉換，最後介紹如何將程式下載到記憶體中。

4.1 軟體編譯流程

圖 4.1 參考計算機組織教科書[13]的內容，其講述了整個編譯的流程，程式發開者撰寫好 C 語言的 Source code 後，經過編譯器編譯會產生組合語言程式，組合語言再經過組譯器後會產生目的檔(Object file)，Object file 裡面包含了機械碼和一些尚未連結的函式，尚未連結的函式將以 Symbol 的方式暫時存在，連結器會負責將這些 Source code 產生的 Object file 和 Library 的 Object file 連結在一起，重新編排每道指令的記憶體位置，最後生成可執行檔，再透過 Loader 載入到記憶體中。

在 2.1.3 小節軟體開發環境中有提到編譯器工具鏈使用的是 RISC-V GNU Compiler Toolchain，其編譯指令均可在 GNU GCC 手冊中[14]查找，將不在此贅述。

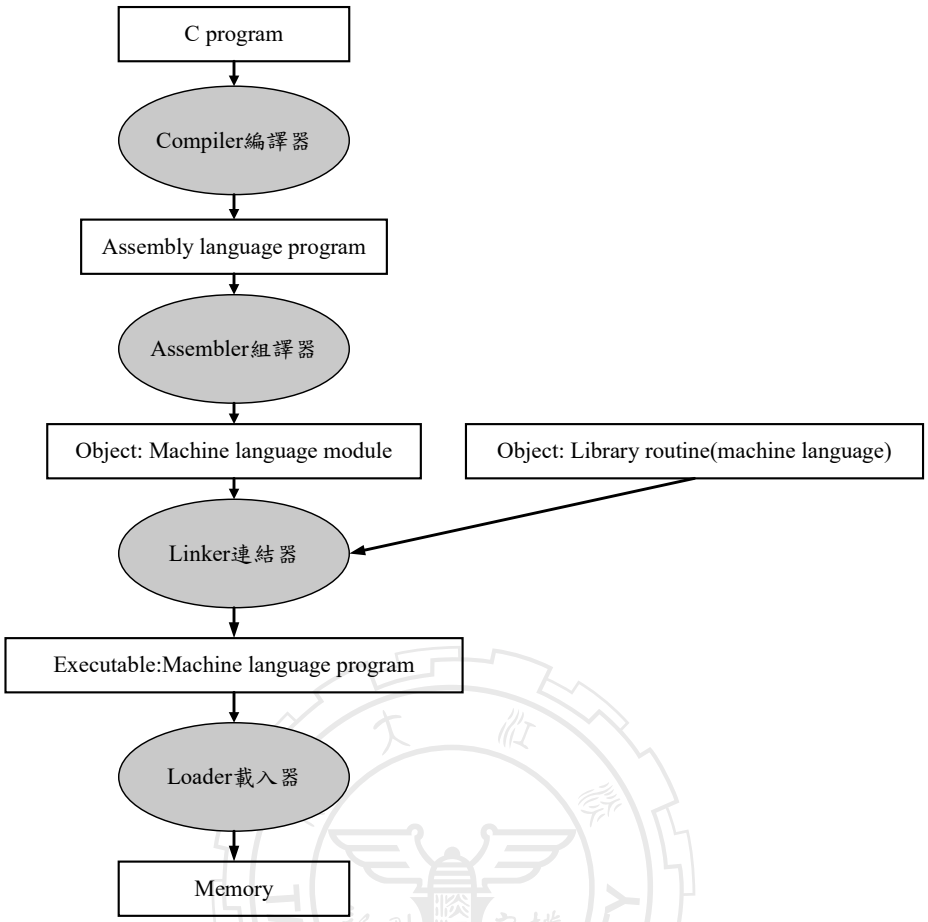


圖 4.1、軟體編譯流程[13]

4.2 周邊裝置記憶體映射地址

如圖 3.22 所示，Qsys 分配了每個周邊在 CPU 定址空間中的基底地址，透過讀寫該周邊的地址來操作周邊控制器中暫存器的數值，進而達到控制周邊裝置，其地址可以由巨集(Macro)的方式定義成符號(Symbol)，在撰寫程式的時就可以 Symbol 的方式來開發，編譯時前處理器才會換成真實的地址資訊，而定義成 Symbol 這件事，可以由 Intel Nios II EDS(Embedded Development Suite)工具來幫你產生定義[15]，也可以由程式開發者手動定義，但當周邊裝置愈加繁多時，由手動定義周邊的基底地

址將顯得捉襟見肘。

本小節將針對使用 Intel Nios II EDS 工具做示範，其流程如圖 4.2 所示，當 Qsys 產生硬體電路的 RTL Code 時，同時也記錄了 SOPC 的資訊，其中有一項資訊就包含了周邊裝置的基底地址，這些資訊都記錄在專案資料夾裡的.sopcinfo 的檔案，打開 Nios II Command Shell 找到此檔案路徑後下 sopc-create-header-file 命令，就會產生出的含有基底地址定義的 header file，實際操作如圖 4.3 所示，部分資訊如圖 4.4 所示，其數值均可與圖 3.22 做比對。

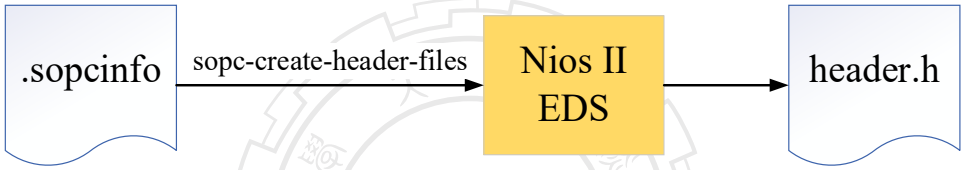


圖 4.2、sopcinfo 轉檔流程

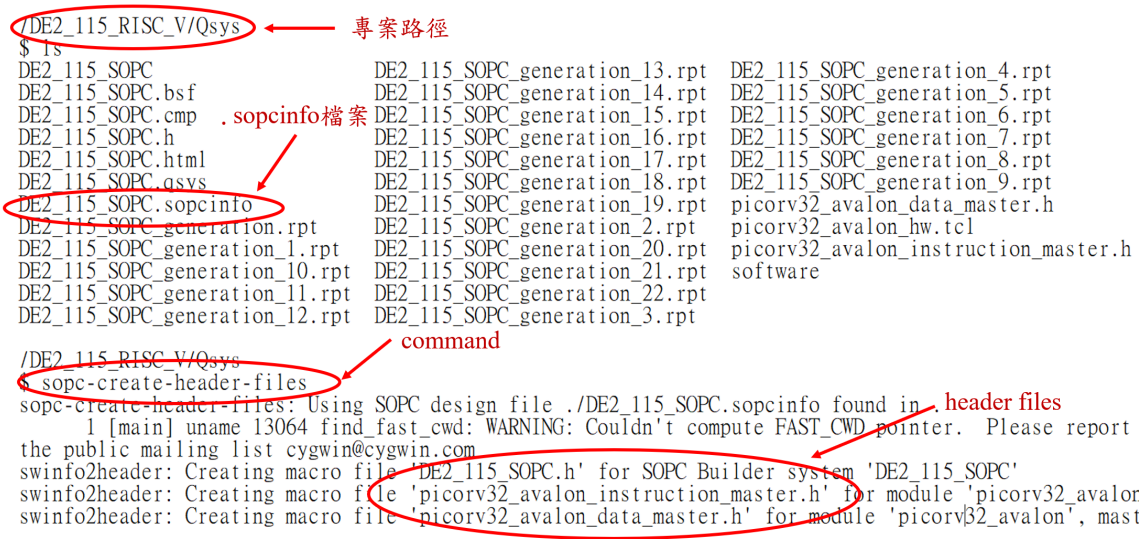


圖 4.3、sopcinfo 轉檔操作

```
#define PICORV32_AVALON_DATA_MASTER_BUTTON_COMPONENT_TYPE altera_avalon_pio
#define PICORV32_AVALON_DATA_MASTER_BUTTON_COMPONENT_NAME button
#define PICORV32_AVALON_DATA_MASTER_BUTTON_BASE 0x0
#define PICORV32_AVALON_DATA_MASTER_BUTTON_SPAN 16
#define PICORV32_AVALON_DATA_MASTER_BUTTON_END 0xf


#define PICORV32_AVALON_DATA_MASTER_HEX7_COMPONENT_TYPE altera_avalon_pio
#define PICORV32_AVALON_DATA_MASTER_HEX7_COMPONENT_NAME HEX7
#define PICORV32_AVALON_DATA_MASTER_HEX7_BASE 0x10
#define PICORV32_AVALON_DATA_MASTER_HEX7_SPAN 16
#define PICORV32_AVALON_DATA_MASTER_HEX7_END 0x1f


#define PICORV32_AVALON_DATA_MASTER_LED_COMPONENT_TYPE altera_avalon_pio
#define PICORV32_AVALON_DATA_MASTER_LED_COMPONENT_NAME LED
#define PICORV32_AVALON_DATA_MASTER_LED_BASE 0xa0
#define PICORV32_AVALON_DATA_MASTER_LED_SPAN 16
#define PICORV32_AVALON_DATA_MASTER_LED_END 0xaf


#define PICORV32_AVALON_DATA_MASTER_SW_COMPONENT_TYPE altera_avalon_pio
#define PICORV32_AVALON_DATA_MASTER_SW_COMPONENT_NAME SW
#define PICORV32_AVALON_DATA_MASTER_SW_BASE 0xb0
#define PICORV32_AVALON_DATA_MASTER_SW_SPAN 16
#define PICORV32_AVALON_DATA_MASTER_SW_END 0xbf


#define PICORV32_AVALON_DATA_MASTER_SRAM_COMPONENT_TYPE altera_avalon_onchip_memory2
#define PICORV32_AVALON_DATA_MASTER_SRAM_COMPONENT_NAME sram
#define PICORV32_AVALON_DATA_MASTER_SRAM_BASE 0x8000000
#define PICORV32_AVALON_DATA_MASTER_SRAM_SPAN 32768
#define PICORV32_AVALON_DATA_MASTER_SRAM_END 0x8007fff
```

圖 4.4、header file 部分資訊

4.3 連結器腳本修改

在軟體編譯流程中有提到連結器所扮演的角色，Object file 在尚未連結前，裡面的機械碼並無分配實際的記憶體地址，甚至一些函式的機械碼根本不在裡面，僅是以符號的形式存在，需要進行連結(Linking)和重定位(Relocation)，連結函式庫以及修改程式在載入記憶體時的地址。

Picorv32 GitHub 網站提供了兩個連結器腳本，一個負責連結函式庫 Linking 的部分，另外一個則是負責 Relocation 的部分，其關係如圖 4.5 所示。

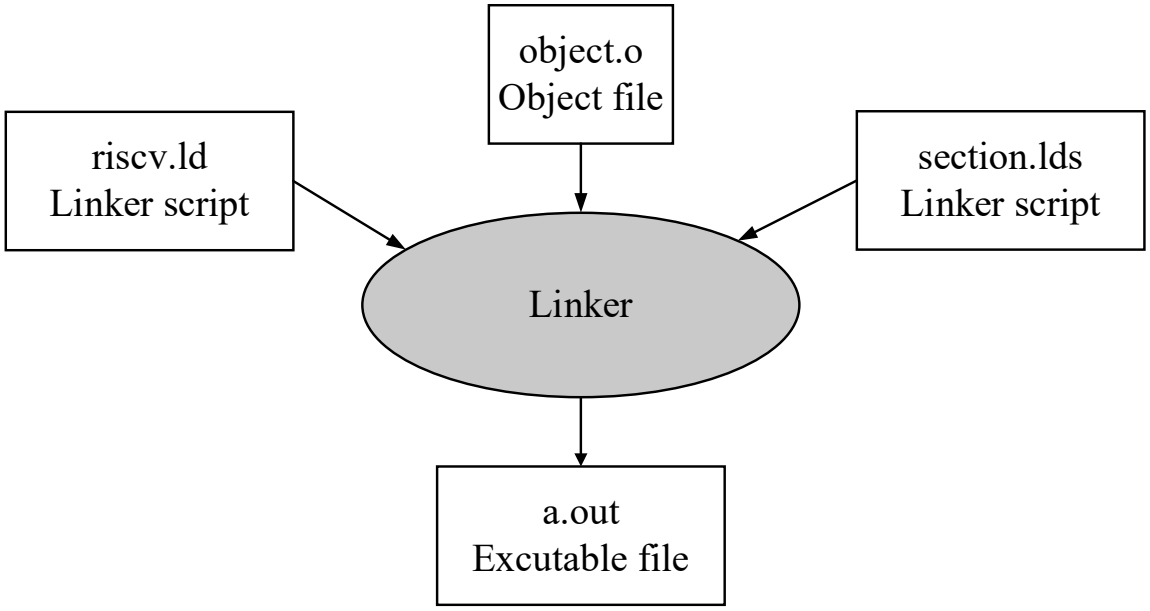


圖 4.5、連結器和連結器腳本

riscv.ld 負責連結函式庫和定義程式進入點，而 section.lds 則負責程式的重定位，兩者都需要稍微修改，函式庫的內容並沒有做更動，也沒有加入新的函式庫，所以僅需定義程式進入點即可，而 Relocation 的部分則是為了要讓程式能夠正確的擺放在記憶體中的正確地址，所以也要對 section.lds 進行修改。

在圖 3.22 中提到 On-Chip Memory 的記憶體地址為 0x0800_0000~0x0800_8000，需要正確的將程式擺放在這個記憶體區段，並且程式的進入點為 0x0800_0000，如圖 4.6 所示，修改程式進入點和起始地址，如同先前圖 3.14 中提到的一樣，這是需要軟硬體共同配合的。

在 section.lds 連結器腳本中定義記憶體的種類、執行權限和記憶體起始地址跟大小如圖 4.7 所示，RAM(xrw)的意思是這塊記憶體是可執行、可讀、可寫，若像是一些 Flash 可能只能讀不能寫，定義完記憶體後即可開始定義要把哪個節區的資料放到 RAM 中，由於實驗中的記憶體只有使

用 On-Chip memory，所以所有的資料都要放到 RAM 中，如 text、data、bss、heap 等，如圖 4.7 所示將 text 節區的資料放入 RAM 中，其他節區的資料亦如法炮製。

```
OUTPUT_FORMAT("elf32-littleriscv", "elf32-littleriscv",
              "elf32-littleriscv")
OUTPUT_ARCH(riscv)
ENTRY(_start)                    Program entry point
SECTIONS
{
 . = 0x08000000;                 _start address
 .text           :
 {
  *(.text)
  *(.text.unlikely .text.*_unlikely .text.unlikely.*)
  *(.text.exit .text.exit.*)
  *(.text.startup .text.startup.*)
  *(.text.hot .text.hot.*)
  *(.stub .text.* .gnu.linkonce.t.*)
  /* .gnu.warning sections are handled specially by elf32.em.  */
  *(.gnu.warning)
 }
```

圖 4.6、修改 riscv.ld 程式進入點

```
MEMORY
{
   RAM (xrw)      : ORIGIN = 0x08000000, LENGTH = 0x8000
}

SECTIONS {
   /* The program code and other data goes into RAM*/
   .text :
   {
      . = ALIGN(4);
      *(.text)          /* .text sections (code) */
      *(.text*)         /* .text* sections (code) */
      *(.rodata)        /* .rodata sections (constants, strings, etc.) */
      *(.rodata*)       /* .rodata* sections (constants, strings, etc.) */
      *(.srodata)       /* .rodata sections (constants, strings, etc.) */
      *(.srodata*)      /* .rodata* sections (constants, strings, etc.) */
      . = ALIGN(4);
      _etext = .;       /* define a global symbol at end of code */
      _sidata = _etext;  /* This is used by the startup in order to initialize the .data secion */
   } >RAM
```

Memory Base Address and Length

Put these sections into RAM

圖 4.7、修改 sections.lds

當連結完後會產生可執行檔，透過 gcc 的二進制實用工具[16]objdump 來顯示執行檔的內容，確認程式的起始地址和程式進入點是否為 0x08000000 和_start，如圖 4.8 所示，程式進入點確實是_start 而起始地址也確實是 0x08000000，以上就完成了連結器腳本的修改。

```
~/Desktop$ riscv32-unknown-elf-objdump -D a.out

a.out:     file format elf32-littleriscv


Disassembly of section .text:

08000000 <_start>:
 8000000:       00001197                auipc   gp,0x1
 8000004:       45018193                addi    gp,gp,1104
 8000008:       82818513                addi    a0,gp,-2008
 800000c:       82818613                addi    a2,gp,-2008
 8000010:       40a60633                sub     a2,a2,a0
```

程式進入點和起始地址

圖 4.8、程式進入點和起始地址

4.4 執行檔案的格式轉換

編譯過後的可執行檔為 ELF 格式，內容含有表頭資訊如圖 4.9 所示，可以透過 objcopy 工具將其轉換成.bin 或是.hex 檔案格式如圖 4.10 所示，去除表頭將取得更純粹的機械碼，方便進行後續的記憶體初始化和程式下載。

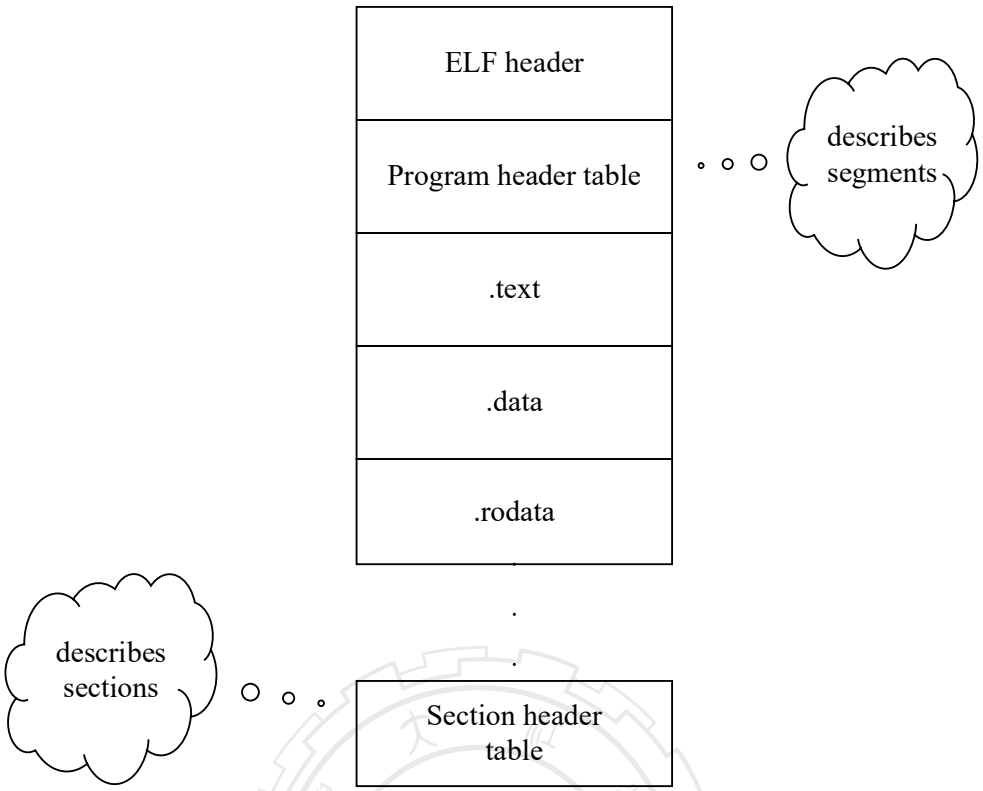


圖 4.9、ELF 檔案格式[17]

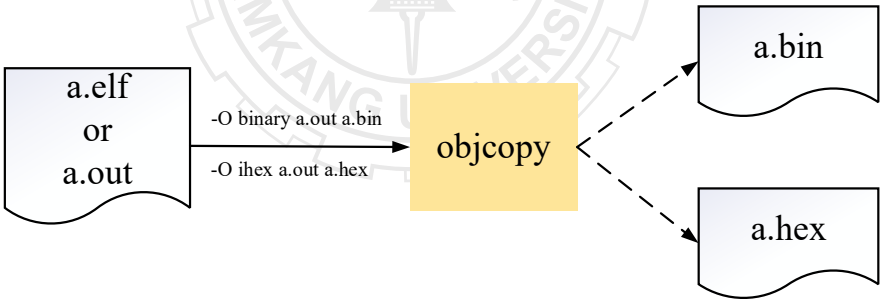


圖 4.10、ELF 檔案格式轉換

下文將針對 Intel Hexadecimal Object File 格式說明，本論文用此檔案格式來初始化 FPGA 上的 On-Chip memory。

ihex 檔案格式是 Intel 於 1988 年提出的一種檔案格式[18]，為了 8-bit, 16-bit 和 32-bit 的處理器而設計，此檔案描述了 16 進制的 object file

格式，用來當作是可程式化唯讀記憶體(Programmable Read-Only Memory, PROM)或是其他硬體模擬器的輸入檔案，是以 ASCII Code，文本的方式來儲存資料，其一般紀錄格式如圖 4.11 所示，以冒號當作記錄的開頭標記，接著是紀錄資料的長度、記憶體地址偏移值，紀錄的型態、資料和校驗碼。

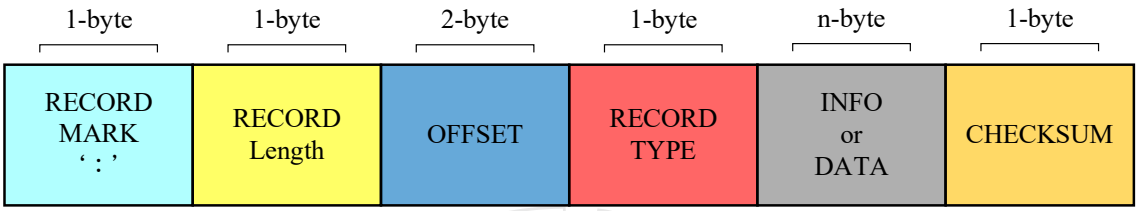


圖 4.11、General Record Format

記錄型態(Record Type)如表 4.1 所示，00 代表紀錄的是一般的資料，01 代表是檔案的結尾，02 代表紀錄的是擴展區段地址，03 代表紀錄的是起始區段地址，04 代表記錄的是擴展線性地址，05 代表紀錄的是起始線性地址。

表 4.1、RECORD TYPE

RECORD TYPE	Description
00	Data Record
01	End of File Record
02	Extended Segment Address Record
03	Start Segment Address Record
04	Extended Linear Address Record
05	Start Linear Address Record

以下講解一個範例，範例資料如圖 4.12 所示：解讀第一行資料時，先觀察 RECORD TYPE 為 04，代表這是擴展線性地址紀錄的格式，擴展線性地址紀錄格式如圖 4.13 所示，擴展線性地址是為了記錄線性基底地址的 16~32 位元，開頭的 02 代表紀錄的資料只有 2 個 Byte，LOAD OFFSET 在擴展線性地址紀錄格式中為定值 0000 不用理會，紀錄資料 0800 代表高 16 位元線性基底地址(Upper Linear Base Address, ULAB)為 0x0800，校驗碼為 F2；

```
1:020000040800F2
2:10000000971100009381014513858182138681828 7
3:10001000306A64093050000EF00803C17050000062
4:10002000130505706308050017050000130585693 1
5:10003000EF00C06EEF00404D03250100930541002 5
6:10004000130600 0EF0000076F004047130101FF97
7:10005000232481008 3C7C18023261100639207 2F5
8:10006000930700006 8A0700371500081305458 1D0
9:10007000970000 0F70000009307100023 86F1803E
10:100080008320C100032481001301010167800000 67
```

圖 4.12、Hex File 範例

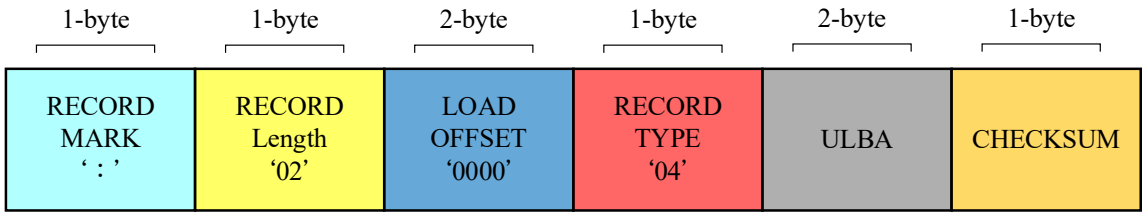


圖 4.13、Extended Linear Address Record

解讀第三行的資料時，先觀察 RECORD TYPE 為 00 可以知道紀錄的是一般資料，開頭的 10 代表紀錄的資料有 0x10 個 Byte 也就是 16 個 Byte 的資料，LOAD OFFSET 為 0010 代表偏移值為 0x0010，配合第一行的 ULAB 就可以算出這行的記憶體寫入地址為 0x0800_0010，中間 3306A64093050000EF00803C17050000 為寫入記憶體的 16 Byte 資料，最後 62 為校驗碼。

4.5 程式下載和記憶體初始化

本小節提到了三種下載程式到記憶體的方式:

第一種是將 ELF 執行檔轉換成 Hex File 連同硬體電路的 RTL Code 一起在 Quartus II 上合成以 SOF 檔案來配置 FPGA，在使用 Qsys 設計 SOPC 時就必須指定 Hex File 的路徑如圖 4.14 所示，當配置完 FPGA 時，On-Chip Memory 就會按照 Hex File 的內容來初始化，FPGA 上的 On-Chip Memory 資源通常並不多，這是一種非常消耗資源的使用方式，因此通常會將程式儲存在外部記憶體如 SDRAM、FLASH、EEPROM 等記憶體。

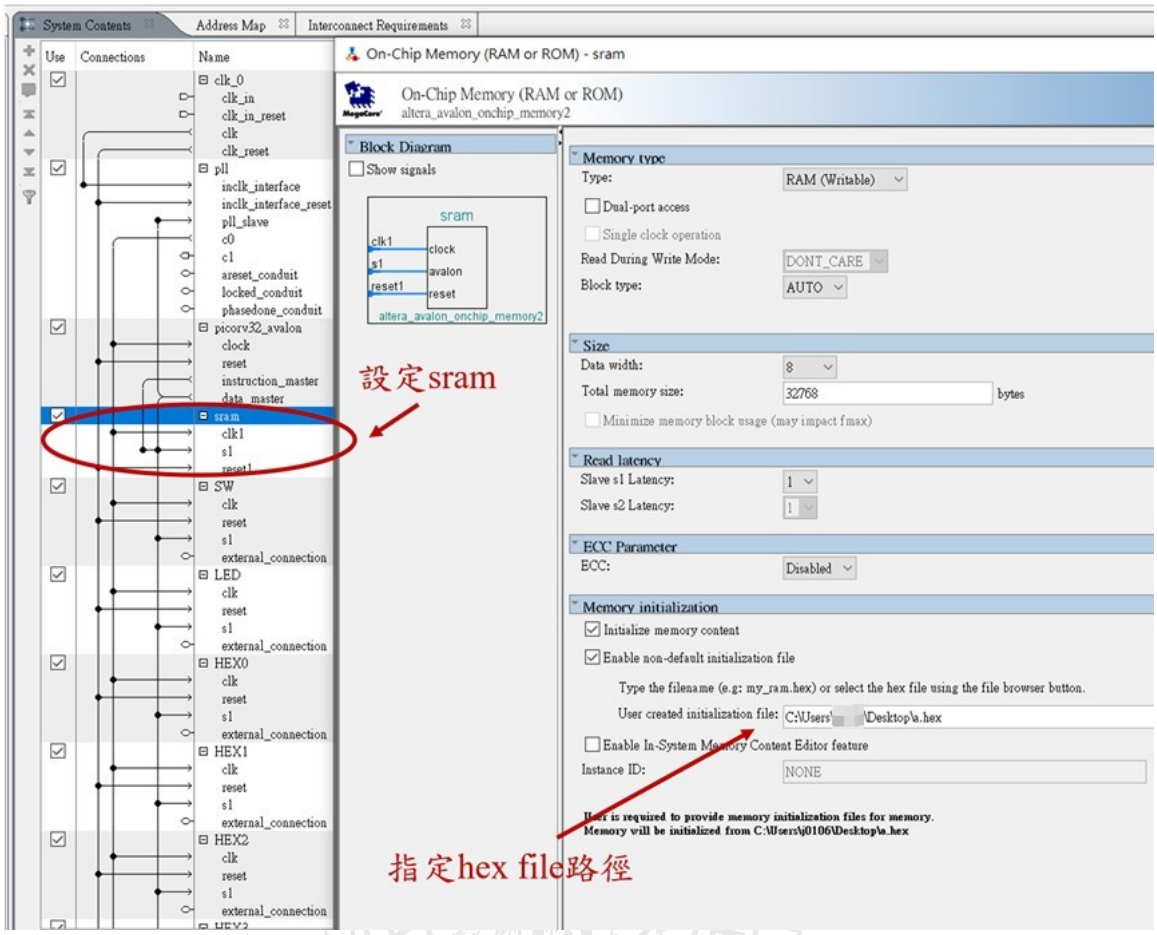


圖 4.14、Hex File 初始化記憶體

在實驗中需要注意的一點是，On-Chip Memory 會以自己的基底地址作為初始化的起始地址如圖 4.15 所示，因此當 Hex File 裡描述的資料是要從 0x0800_0000 開始載入時會發生錯誤，On-Chip Memory 會認為是要從(On-Chip Memory 基底地址+0x0800_0000)開始載入資料，但其實應該從 On-Chip Memory 基底地址開始載入資料，面對這種情況，可以透過 objcopy 更改 Hex File 裡面資料的地址如圖 4.16 所示，在記憶體初始化時就能剛好的把資料擺在正確的地址。

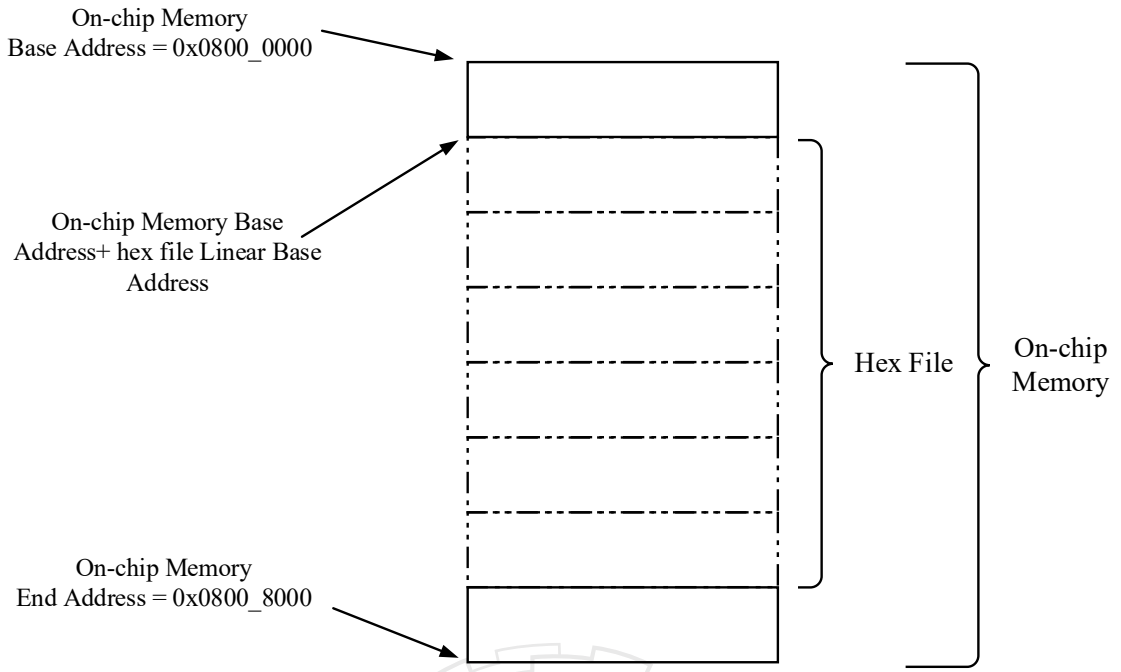


圖 4.15、Hex File 初始化 On-Chip Memory

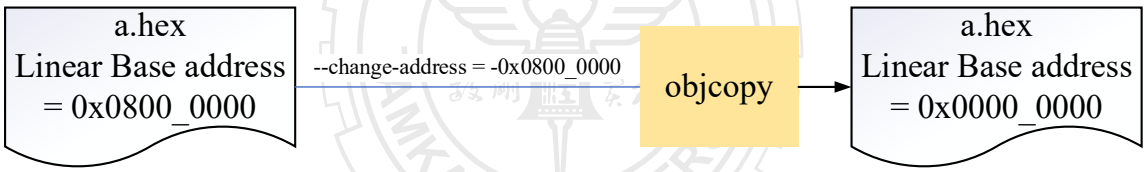


圖 4.16、更改 Hex File 資料地址

第二種下載程式到記憶體的方式，須配合自製的 Debug module 來做韌體程式開發，其流程如圖 4.17 所示。大致分成以下三個步驟：

第一步：傳送燒錄請求，這相當於傳送一個密碼，當硬體檢測到燒錄請求後會發送 reset 信號給 Picorv32 處理器並且進入燒錄模式等待後續的資料傳入。

第二步：傳送記憶體燒錄的起始地址和資料長度。

第三步：傳送串流資料，直到資料傳完為止。

上述每個步驟完成時，硬體都會驗證校驗碼並傳回一個 Byte 的校驗資料也就是 error_code，依此來判斷此步驟是否成功，若失敗則從頭來過，重新開始發送燒錄請求。

在硬體上並未做過多的檢查和容錯機制，每個階段性任務結束後就會回傳 error_code，當韌體收到 error_code 為不成功時，就重新啟動燒錄程序，也可以設定看門狗計時器(Watchdog timer, WDT)當上傳程式卡住時自動重新執行，此方法可簡化硬體設計的複雜度，也減少硬體資源消耗。

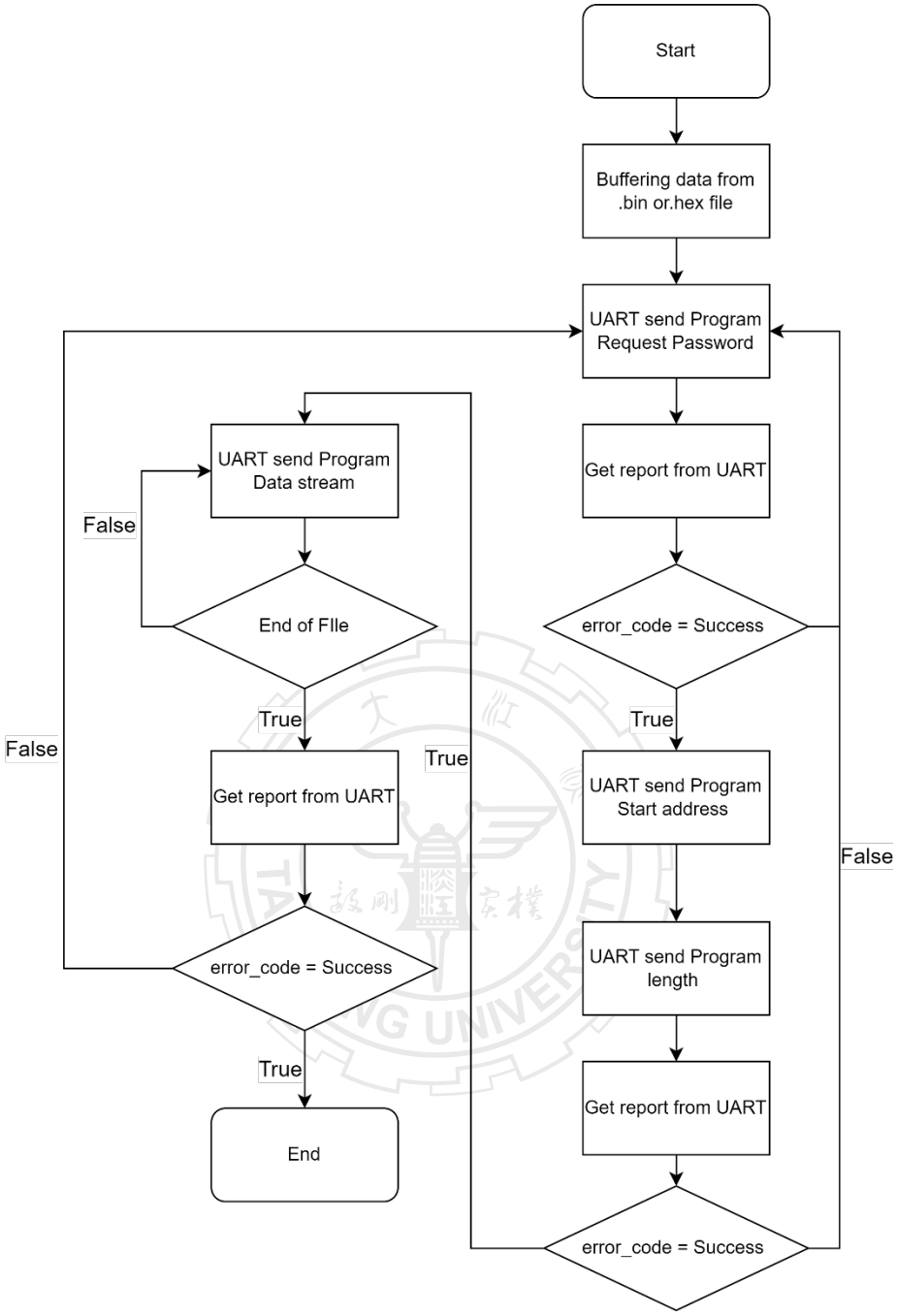


圖 4.17、Debug module 韌體程式流程圖

第三種方式是透過叫做 nios2-download[19]的 Nios II EDS 工具來下載程式到記憶體，在 Debug module 設計中有提到，替換掉系統中的 Nios II 處理器將無法使用它自帶的 JTAG Debug module，但其實還是有辦法

的，其流程如圖 4.18 所示。

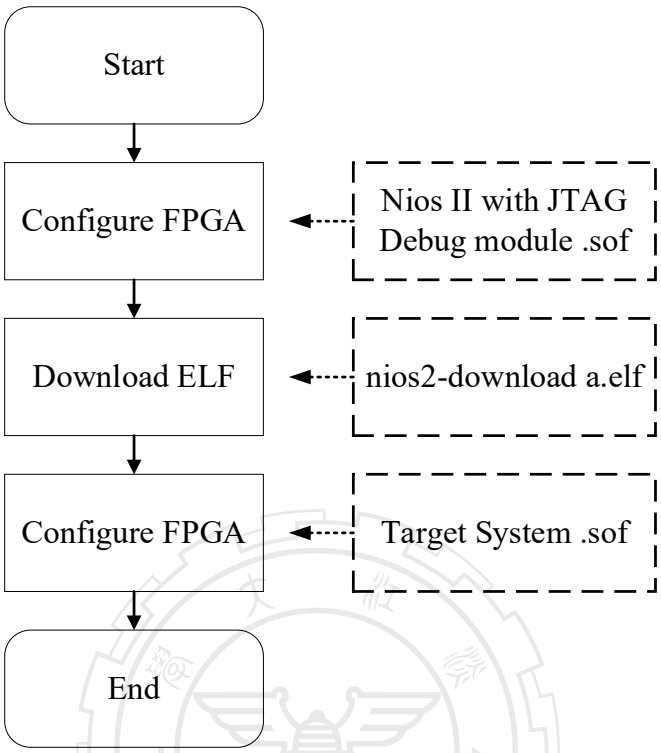


圖 4.18、使用 Nios II EDS 燒錄流程

先配置 FPGA 成為一個 Nios II 程式下載器如圖 4.19 所示，再透過此下載器和 nios2-download 工具來下載程式到記憶體中，最後重新配置 FPGA 成為目標系統即可運行，整個配置 FPGA 的過程不斷電，因此揮發性記憶體如 SDRAM、SRAM 也能完整地保存資料，此方法只需要更換記憶體控制器，即可下載到不同類型的記憶體，將圖 4.18 的流程圖寫成腳本(Script)文件，將會大幅縮短程式的開發週期。

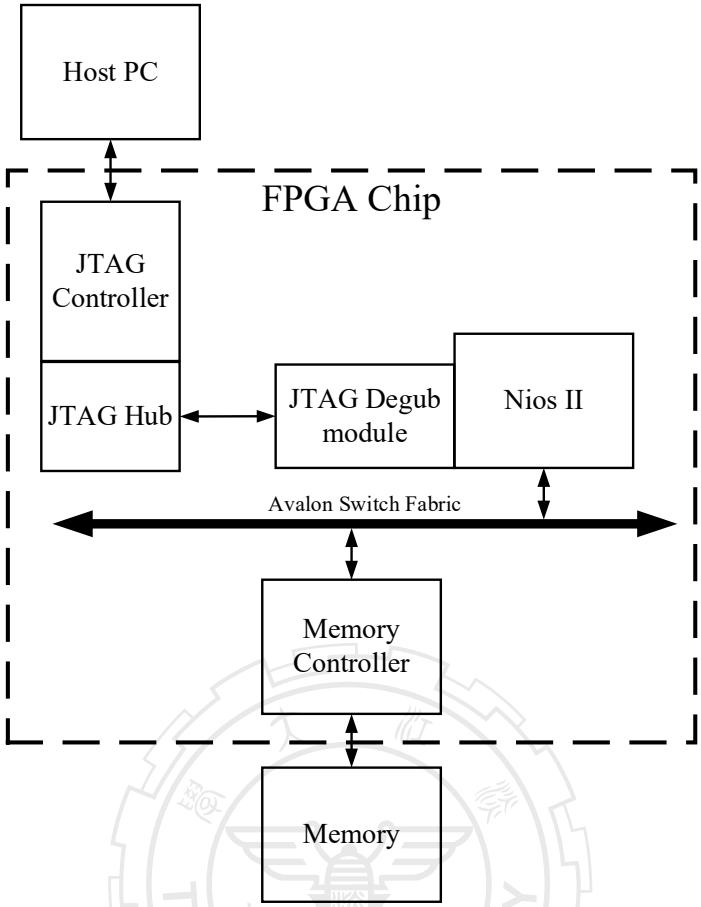


圖 4.19、Nios II 程式下載器

注意，nios2-download 只能識別 ELF 檔案格式文件，但這並不代表不能下載其他檔案格式的資料到記憶體中，只要能夠把目標文件轉換後塞進 ELF 檔案裡即可。

三種下載程式的方法適用的場景各有不同，使用 Hex File 初始化 On-Chip Memory 的方法適合用在程式小的應用，因為 On-Chip Memory 在 FPGA 上是稀少的資源，無法儲存大型程式；使用自製 Debug module 下載程式能有更好的移植性，換到不同廠牌的 FPGA 上也能適用；使用 Nios II JTAG Debug module 來幫助程式下載不僅速度快也很方便，缺點是無法使用在其他廠牌之 FPGA 上。

這三種下載方式可以歸類為如下表 4.2 所示，本文推薦，若是使用 Intel 的 FPGA，以 Nios II JTAG Debug module 來下載程式是最快最方便的。

表 4.2、程式下載方式比較

下載程式方式	適用場景或優勢	外掛 USB to TTL 轉接器
Hex File 初始化 On-Chip Memory	小型應用	不需要
自製 Debug module	移植性佳	需要
Nios II JTAG Debug module	方便快速	不需要

第五章 實驗結果

本章節將對 SOPC 上的周邊元件進行展示和驗證，包含了基礎 IO 的操作，以及 JTAG UART 的通信，和使用 Nios II Debug module 下載程式到外部的 SDRAM 等三種實驗結果，最後對 Picorv32 處理器和 Nios II 處理器進行了資源消耗以及性能的比較。

在 2.2 節系統架構中有提到使用自製的 Debug module 來下載程式到 SDRAM、Flash 等外部記憶體，由於尚未實作出 Host PC 端的下載程式，因此本章節並無提供相對應的展示，僅在 3.4 節 Debug module 設計與 4.5 節程式下載和記憶體初始化中提供軟硬體的設計方法。

5.1 實驗一：基礎 IO 測試和 Demo

加入 IO 周邊裝置如 LED 燈、七段顯示器、指撥開關和按鈕開關進行讀寫測試，其硬體 SOPC 架構圖如圖 5.1 所示。

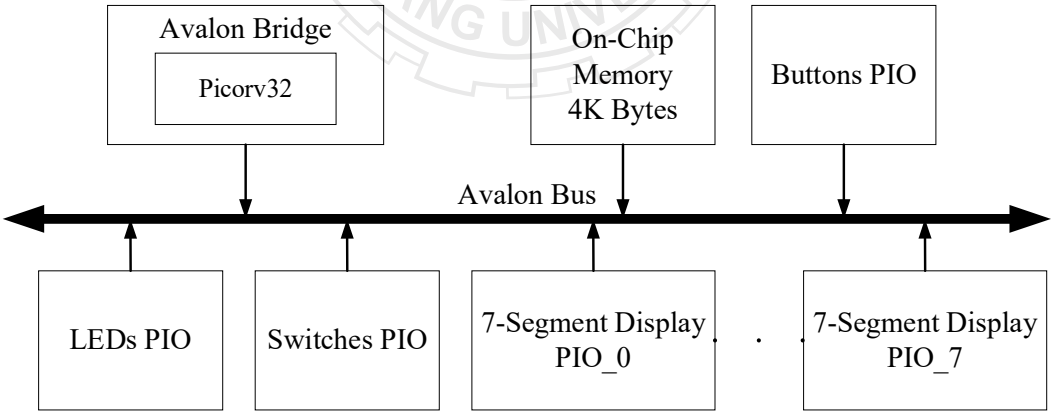


圖 5.1、IO 測試之硬體 SOPC 架構

其完整的軟硬體開發流程如圖 5.2 所示，分成三個部分做開發：

第一部分：在 Quartus II 做 Picorv32 處理器的橋接以及整體的 SOPC 電路合成，最後完成配置 FPGA。

第二部分：在 Qsys 上設計 SOPC，選擇需要的處理器、記憶體和周邊元件,組成我們要的系統,最後由 Qsys 生成 RTL Code 後再回到 Quartus II 上做合成。

第三部分：透過 gcc 編譯器做 C 語言程式的開發，將寫好的 C 語言程式碼編譯出 ELF 可執行檔後透過 objcopy 工具轉換成 On-Chip Memory 的初始化檔案 Hex File，在 Qsys 中呼叫 On-Chip Memory 模組的時候需要指定 Hex File 的檔案位置，才能將 Hex File 連同硬體設計一起打包生成 RTL Code。

開發流程存在相依性關係，為了能夠順利開發，以下為建議的開發順序，分成四個步驟：

步驟一：先完成 C 語言的軟體開發，產生出 ELF 執行檔後轉成 Hex File。

步驟二：回到 Quartus II 做 Picorv32 處理器與 Avalon 介面的橋接。

步驟三：開啟 Qsys 做 SOPC 的設計，並在加入 On-Chip Memory 時指出 Hex File 的檔案路徑用來初始化 On-Chip Memory。

步驟四：回到 Quartus II 做整個系統的合成，配置 FPGA 進行展示。

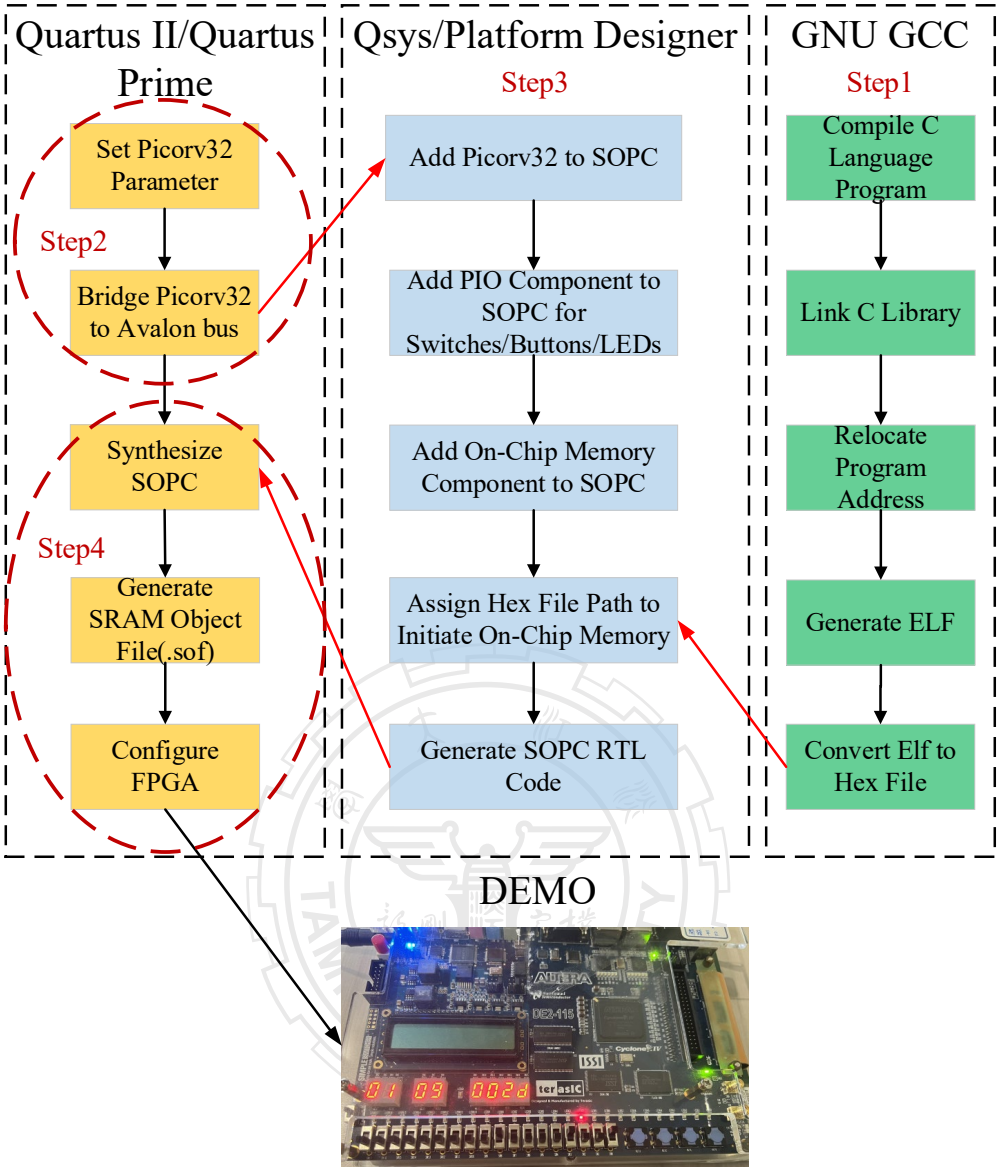


圖 5.2、基礎 IO 測試的軟硬體開發流程

Demo 程式中包含了累加、移位、陣列存取、除法和取餘數等運算。流程圖如圖 5.3 所示，當程式下載到記憶體時，一開始會看到所有七段顯示器歸零，LED 燈不斷循環左移，0、1 號七段顯示器隨著 LED 燈的移位次數而累加數值，4、5、6、7 號七段顯示器顯示的是 15~0 號指撥開關的數值，按鈕開關 KEY[1]、KEY[2]按下時會累加數值並在 2、3 號七段顯

示器上顯示，Demo 程式碼的周邊元件基底地址和副函式定義如圖 5.4 所示，主函式如圖 5.5 所示。

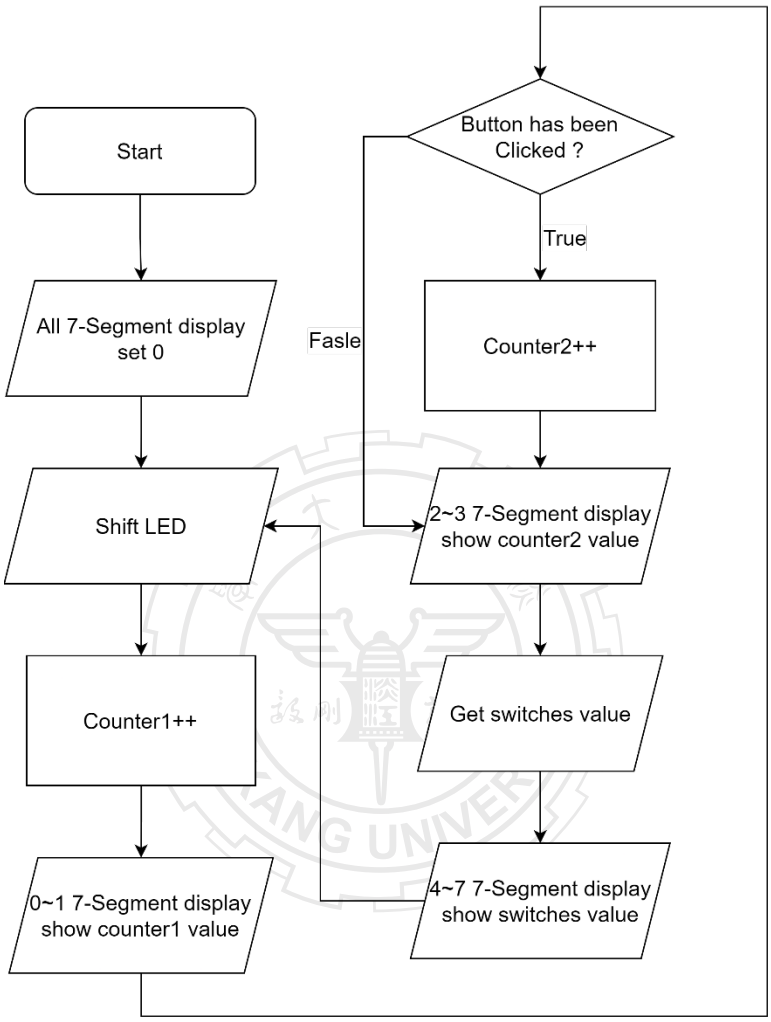


圖 5.3、Demo 程式流程圖

```
#define LED_BASE 0x000000A0
#define HEX0_BASE 0x00000090
#define HEX1_BASE 0x00000070
#define HEX2_BASE 0x00000060
#define HEX3_BASE 0x00000050
#define HEX4_BASE 0x00000040
#define HEX5_BASE 0x00000030
#define HEX6_BASE 0x00000020
#define HEX7_BASE 0x00000010
#define SW_BASE 0x000000B0
#define BUTTON_BASE 0x00000000

unsigned char DEC_7SEG(int number) {
	if (number > 15) {//解碼器最多只能解碼 0 ~ 15，防止七段顯示器誤動作
		return 0xFF;//直接回傳0xFF讓七段顯示器全暗，就不用跑以下的解碼，省時間，也不會記憶體非法存取
	}
	static const unsigned char temp[] = {//宣告一個靜態的常數陣列，陣列大小為16，每個元素的資料寬度為8bit無號數
			0xC0,0xF9,0xA4,0xB0,//共陽型的解碼 0 ~ 15
			0x99,0x92,0x82,0xF8,
			0x80,0x90,0x88,0x83,
			0xC6,0xA1,0x86,0x8E
	};
	return temp[number];
}
void delay(int count){
	int i;
	for(i = 0; i < count; i++);
}
```

圖5.4、周邊元件基底地址和副函式定義

```
int main(void){
	*(int *)(LED_BASE) = 0x0000AAAA;//1010_1010_1010
	*(unsigned char *)(HEX0_BASE) = DEC_7SEG(0);
	*(unsigned char *)(HEX1_BASE) = DEC_7SEG(0);
	*(unsigned char *)(HEX2_BASE) = DEC_7SEG(0);
	*(unsigned char *)(HEX3_BASE) = DEC_7SEG(0);
	*(unsigned char *)(HEX4_BASE) = DEC_7SEG(0);
	*(unsigned char *)(HEX5_BASE) = DEC_7SEG(0);
	*(unsigned char *)(HEX6_BASE) = DEC_7SEG(0);
	*(unsigned char *)(HEX7_BASE) = DEC_7SEG(0);
	delay(500000);
	int i = 0;
	int sw;
	int button_edge;
	int led_cnt = 0x01;
	int hex2_cnt = 0;
	int hex3_cnt = 0;
	while(1){
	 delay(10000);
	 if(led_cnt == 0x02000000){ //0010_0000_0000_0000_0000_0000_0000
		led_cnt = 1;
	 }
	 *(int *)(LED_BASE) = led_cnt;
	 led_cnt = led_cnt << 1;
	 *(unsigned char *)(HEX0_BASE) = DEC_7SEG(i % 16);
	 *(unsigned char *)(HEX1_BASE) = DEC_7SEG(i / 16 % 16);
	 i++;
	 button_edge = *((int *)(BUTTON_BASE) + 3);
	 if((button_edge & 0x01)){ //如果第一顆按鈕開關被按下
		hex2_cnt++;
	 }
	 if(button_edge & 0x02){ //如果第二顆按鈕開關被按下
		hex3_cnt++;
	 }
	 if(button_edge & 0x04){ //如果第三顆按鈕開關被按下
		hex2_cnt = 0;
		hex3_cnt = 0;
	 }
	 *((int *)(BUTTON_BASE) + 3) = 0;//clean button edge_capture
	 sw = *(int *)SW_BASE;
	 *(unsigned char *)(HEX2_BASE) = DEC_7SEG(hex2_cnt);
	 *(unsigned char *)(HEX3_BASE) = DEC_7SEG(hex3_cnt);
	 *(unsigned char *)(HEX4_BASE) = DEC_7SEG(sw & 0x0000000F);
	 *(unsigned char *)(HEX5_BASE) = DEC_7SEG((sw >> 4) & 0x0000000F);
	 *(unsigned char *)(HEX6_BASE) = DEC_7SEG((sw >> 8) & 0x0000000F);
	 *(unsigned char *)(HEX7_BASE) = DEC_7SEG((sw >> 12) & 0x0000000F);
	}
	return 0;
}
```

圖 5.5、IO Demo 主函式

如圖 5.6 所示，七段顯示器正確地顯示了指撥開關的數值，在圖 5.7 中可以確認讀取指撥開關是否正確，可以看到 data_master_address 為 0x000000B0，與指撥開關的基底地址一致，再來確認 data_master_readdata 為 0x00000109，也與指撥開關數值一致。

七段顯示器若要顯示出正確的數值，必須經過解碼如表 5.1 所示，可以透過圖 5.8、圖 5.9、圖 5.10 和圖 5.11 中的 data_master_address 和 data_master_writedata 來確認七段顯示器的數值是否正確，以展示結果和波形驗證來看，確認系統能夠正常運作。

圖 5.6 中左邊四顆七段顯示器顯示本人碩士口試日期。

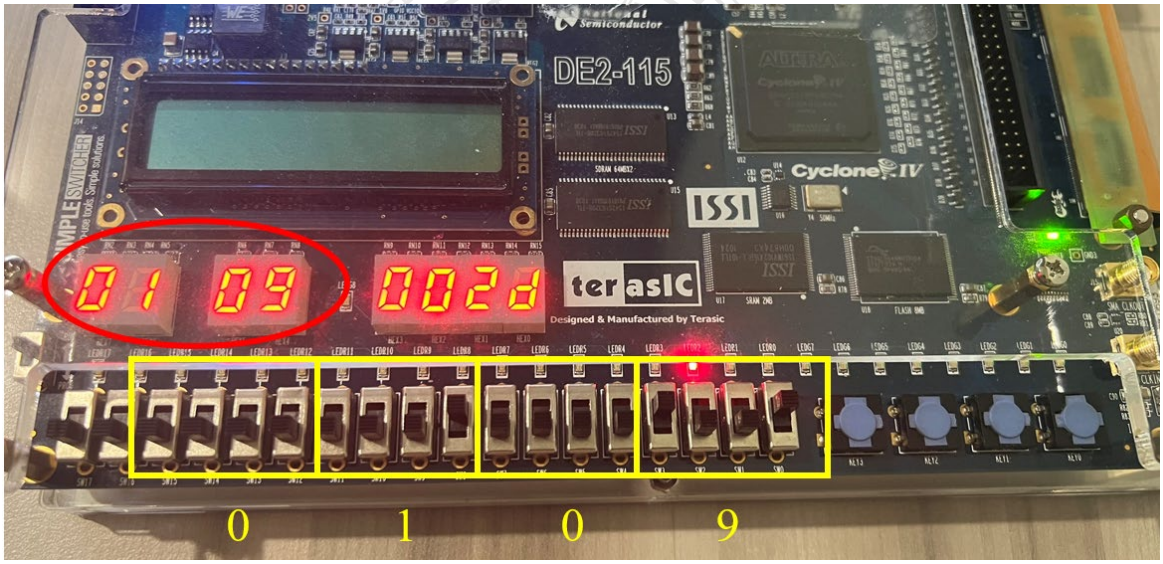


圖 5.6、Demo 範例

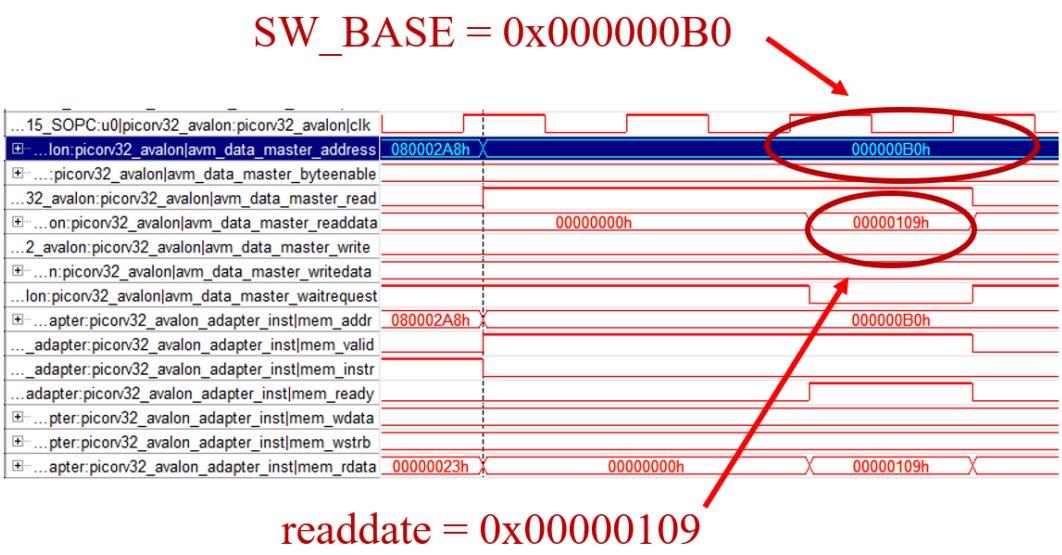


圖 5.7、SW_BASE 讀取波形

表 5.1、共陽極七段顯示器解碼

七段顯示器顯示數值	共陽極七段顯示器解碼數值
0	0xC0
1	0xF9
9	0x90

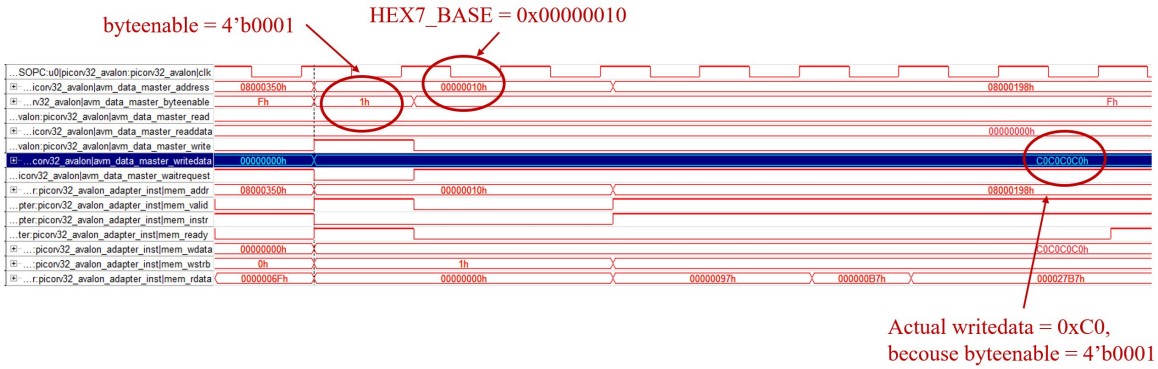


圖 5.8、HEX7_BASE 寫入波形

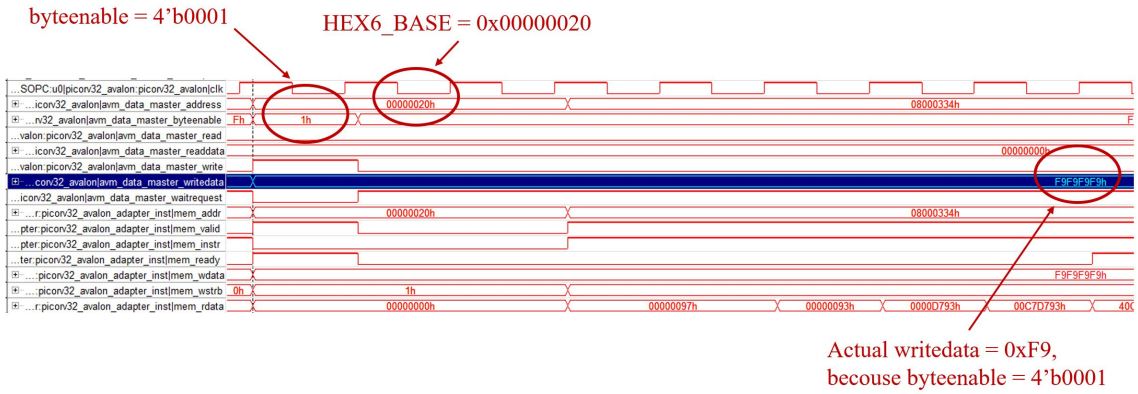


圖 5.9、HEX6_BASE 寫入波形

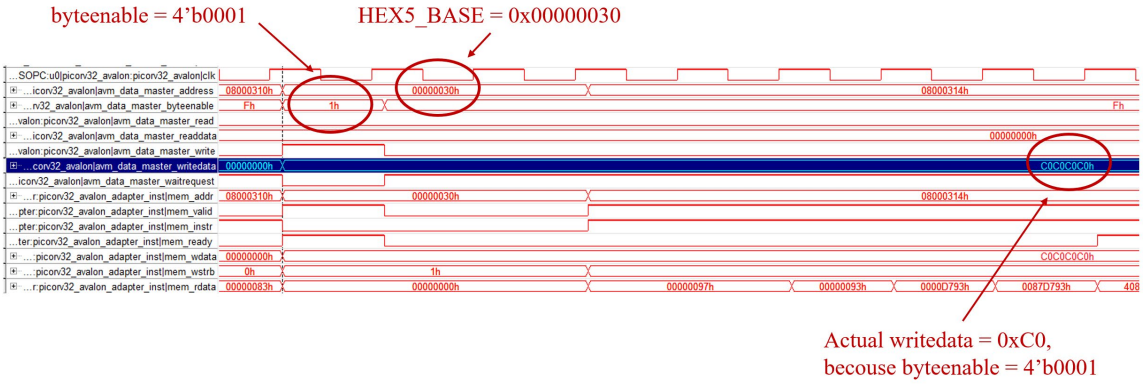


圖 5.10、HEX5_BASE 寫入波形

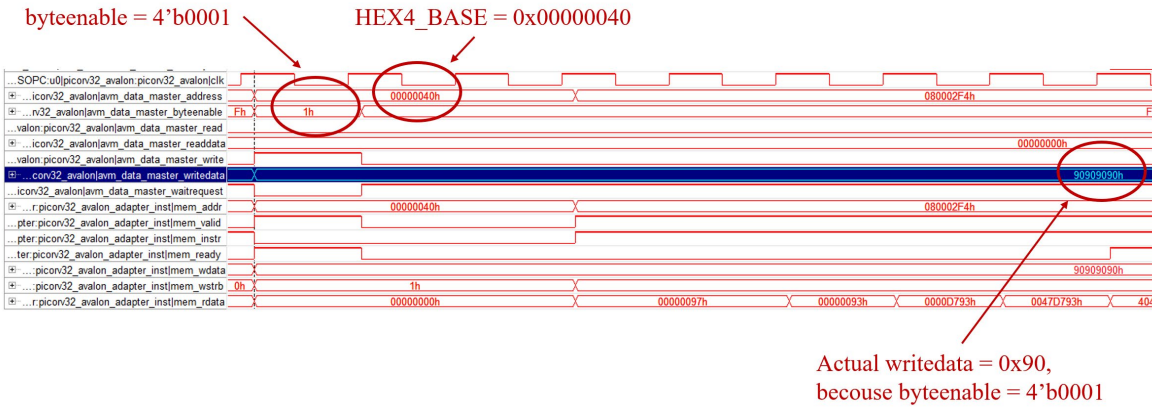


圖 5.11、HEX4_BASE 寫入波形

5.2 實驗二：JTAG UART Demo

JTAG UART Demo 系統架構如圖 5.12 所示，在 Host PC 端 Intel 官方有提供名為 nios2-terminal[19]的接收程式，可以幫助顯示接收到的資料。

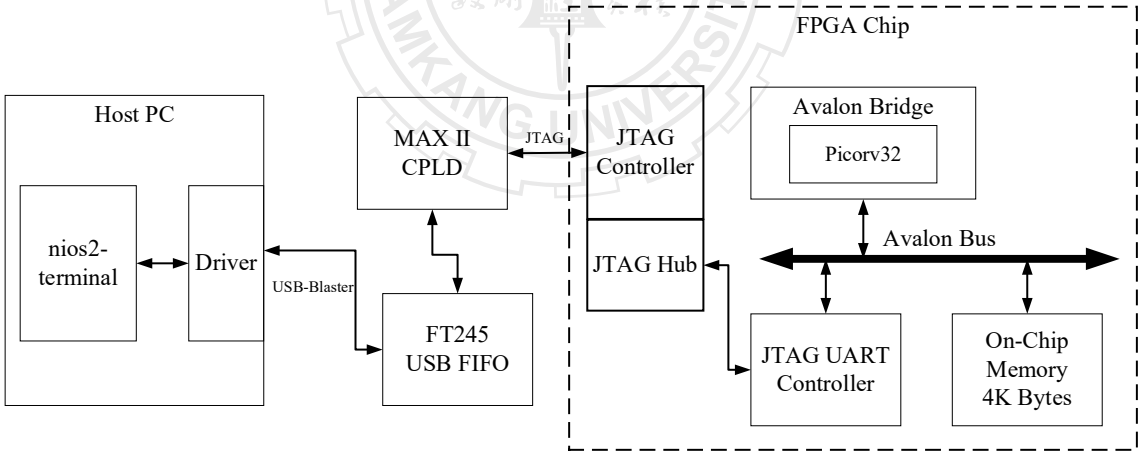


圖 5.12、JTAG UART Demo 系統架構

軟硬體的開發流程如圖 5.13 所示，與實驗一相似，差別在於增加了 JTAG UART Controller 並刪除了用不到的 PIO 元件。

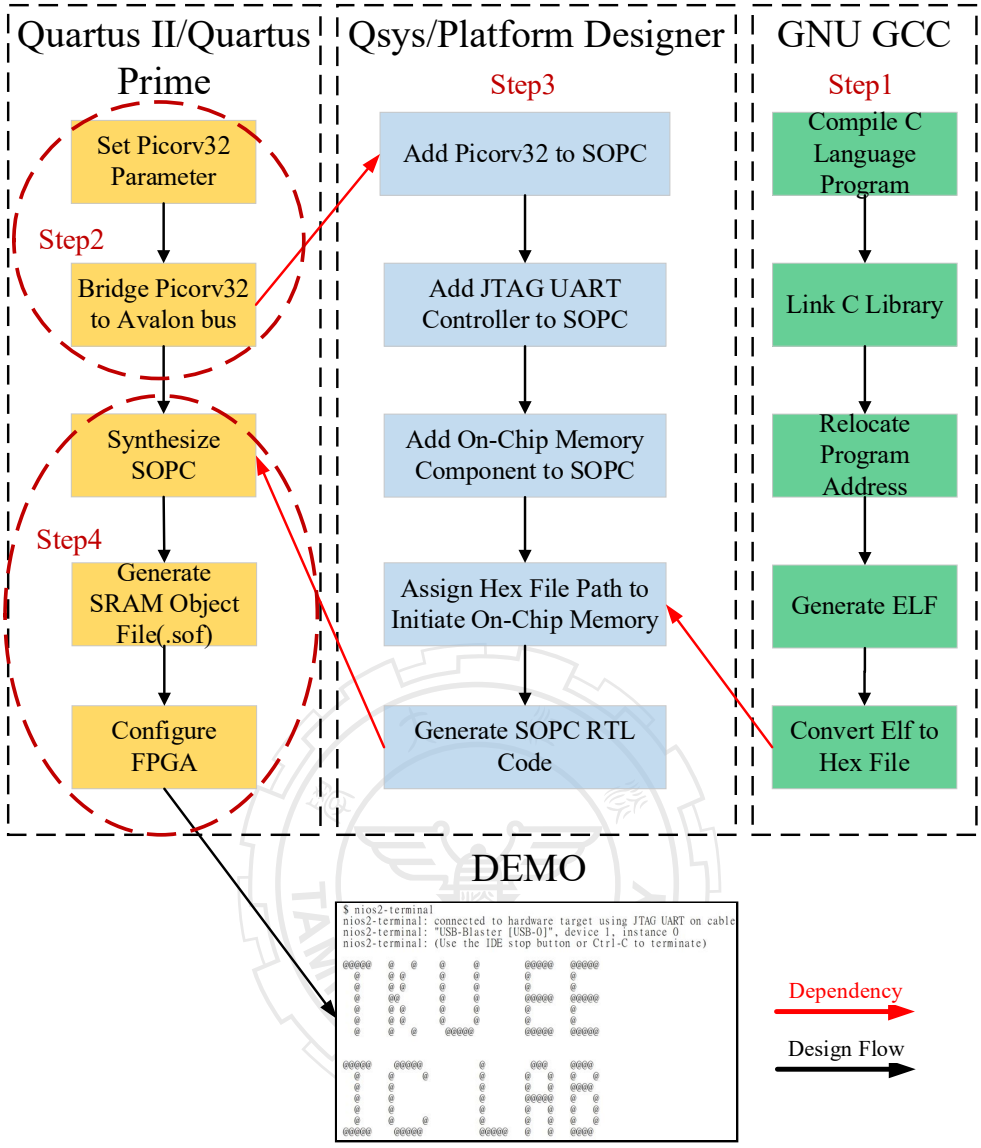


圖 5.13、JTAG UART Demo 軟硬體開發流程

JTAG UART 的暫存器說明如表 5.2 所示，0 號暫存器的 7~0 bits 負責讀寫資料，1 號暫存器的高 16 位元 WSPACE 表示內部 FIFO 還有多少空間可供寫入，這裡僅列出 Demo 需要用到資訊，其他位元暫時不需要關心，如需更詳細的暫存器說明可參考 Intel 的嵌入式 IP 使用手冊[20]。

表 5.2、JTAG UART 的暫存器說明

Offset	Register name	31~16 bits	7~0 bits
0	data	Don't care	DATA
1	control	WSPACE	Don't care

JTAG UART 的基底地址、暫存器和副函式定義如圖 5.14 所示，print_str 函式判斷字元指標的數值是否等於'\0'，若不為'\0'則持續輸出字元到_puts 函式，當遇到'\0'代表字串結束，_puts 函式會判斷 JTAG UART 內部的寫入 FIFO 是否還有寫入空間，再把資料寫入到 JTAG UART 的資料暫存器中，若無空間可供寫入，則會持續等待直至 FIFO 有多餘空間，如果強行寫入資料，可能會導致資料遺失。

```
#define JTAG_UART_BASE 0x000000C0
#define JTAG_UART_DATA *((int *)(JTAG_UART_BASE))
#define JTAG_UART_CTRL *((int *)(JTAG_UART_BASE + 4))


void print_str(char* p) {
        while (*p != '\0') {
                _puts(p++);
        }
}
void _puts(char* p) {
        while ((JTAG_UART_CTRL >> 16 & 0xFFFF) == 0);//no space to write
        JTAG_UART_DATA = *p;
}
```

圖 5.14、JTAG UART 基底地址和副函式定義

主函式如圖 5.15 所示，透過 print_str 函式輸出 TKU EE, IC LAB 圖形文字。

```
int main()
{
    print_str("@@@@@   @   @    @     @   \t@@@@@   @@@@@\n");
    print_str("  @     @ @      @     @   \t@       @    \n");
    print_str("  @     @ @      @     @   \t@       @    \n");
    print_str("  @     @@       @     @   \t@@@@@   @@@@@\n");
    print_str("  @     @ @      @     @   \t@       @    \n");
    print_str("  @     @ @      @     @   \t@       @    \n");
    print_str("  @     @   @     @@@@@    \t@@@@@   @@@@@\n\n\n");

    print_str("@@@@@    @@@@@     \t@        @@@    @@@@  \n");
    print_str("  @     @     @    \t@       @   @   @   @ \n");
    print_str("  @     @          \t@       @   @   @@@@  \n");
    print_str("  @     @          \t@       @@@@@   @   @ \n");
    print_str("  @     @          \t@       @   @   @   @ \n");
    print_str("  @     @     @    \t@       @   @   @   @ \n");
    print_str("@@@@@    @@@@@     \t@@@@@   @   @   @@@@  \n");
    return 0;
}
```

圖 5.15、JTAG UART Demo 主函式

在 Nios II Command Shell 中鍵入 nios2-terminal 打開終端介面，當系統識別到 JTAG UART 有正確連接後即可正常通信，輸出文字如圖 5.16 所示。

```
$ nios2-terminal
nios2-terminal: connected to hardware target using JTAG UART on cable
nios2-terminal: "USB-Blaster [USB-0]", device 1, instance 0
nios2-terminal: (Use the IDE stop button or Ctrl-C to terminate)

@@@@@    @    @     @      @         @@@@@    @@@@@
  @      @ @       @      @         @        @
  @      @ @       @      @         @        @
  @      @@        @      @         @@@@@    @@@@@
  @      @ @       @      @         @        @
  @      @ @       @      @         @        @
  @      @    @      @@@@@           @@@@@    @@@@@


@@@@@     @@@@@          @          @@@     @@@@
  @       @      @       @         @   @    @   @
  @       @             @         @   @    @@@@
  @       @             @         @@@@@    @   @
  @       @             @         @   @    @   @
  @       @      @      @         @   @    @   @
@@@@@     @@@@@        @@@@@     @   @    @@@@
```

圖 5.16、nios2-terminal 顯示 JTAG UART 傳輸文字

5.3 實驗三：使用 Nios II JTAG Debug module 下載程式

本小節示範如何使用 Nios II 的 JTAG Debug module 下載 RISC-V 程式到 SDRAM 中，比對 SDRAM 讀出的內容與 ELF 執行檔的內容是否一致來判斷程式有沒有正確上傳。

實驗流程如圖 5.17 所示，先配置 FPGA 成為程式下載器，接著下載本實驗 RISC-V 測試程式的 ELF 執行檔至記憶體中，最後再以目標系統配置 FPGA 即可順利完成實驗。

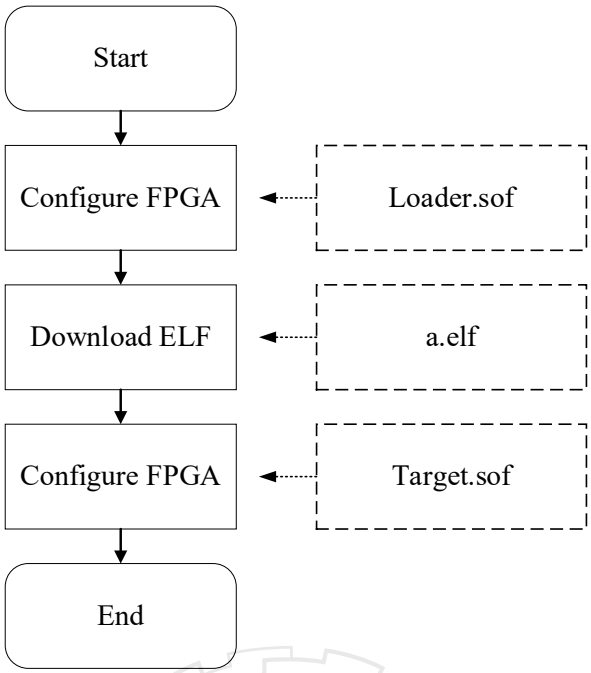


圖 5.17、實驗流程

詳細的實驗步驟如圖 5.18 所示，大致可分為 6 步：

步驟一：在 Quartus II 上合成 Loader SOPC。

步驟二：配置 FPGA 成為 Loader。

步驟三：編譯 RISC-V 的 C 語言程式成 elf 檔。

步驟四：透過 nios2-download 工具下載 elf 檔至 SDRAM 中。

步驟五：在 Quartus II 上合成 Target SOPC。

步驟六：配置 FPGA 成為 Target，即可開始 Demo。

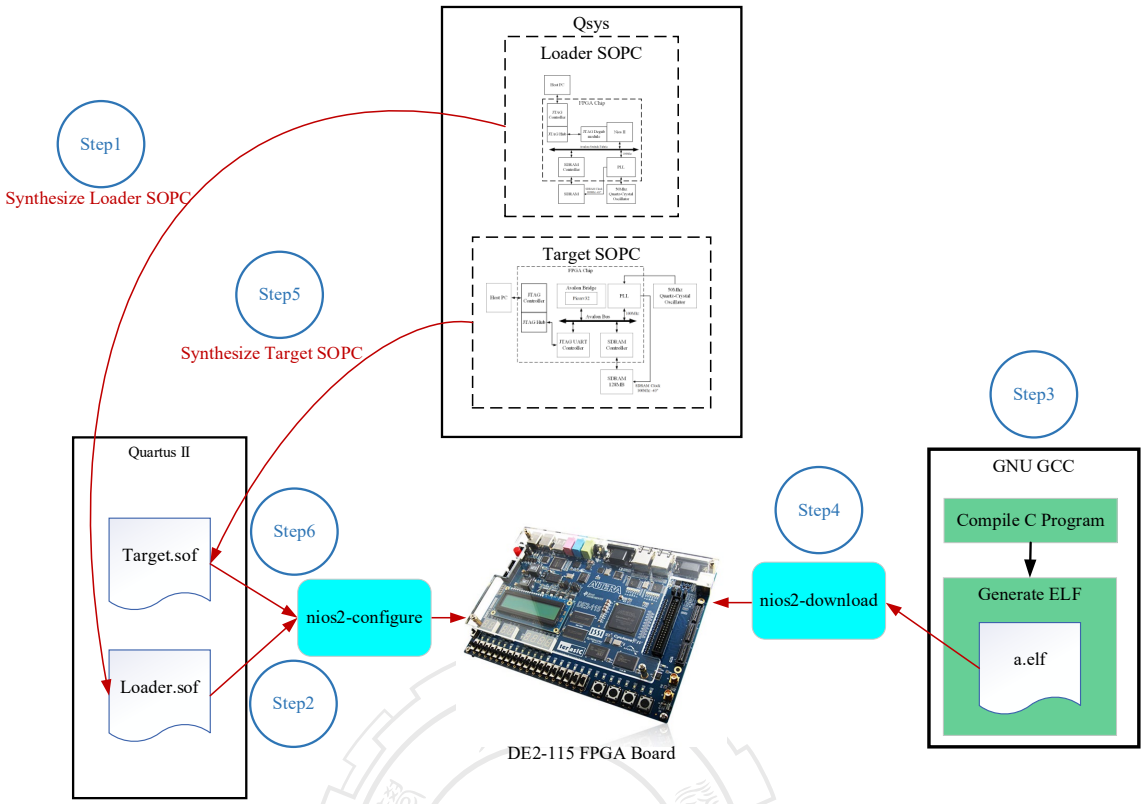


圖 5.18、實驗三實驗步驟

程式下載器架構如圖 5.19 所示，其架構非常簡單，系統中包含了 Nios II with JTAG Debug module, SDRAM Controller, SDRAM, PLL；PLL 以及 SDRAM Controller 的參數設定將會在目標系統中一併介紹。

JTAG Debug module，會自動連接上 JTAG Hub 使用者無須手動連接。

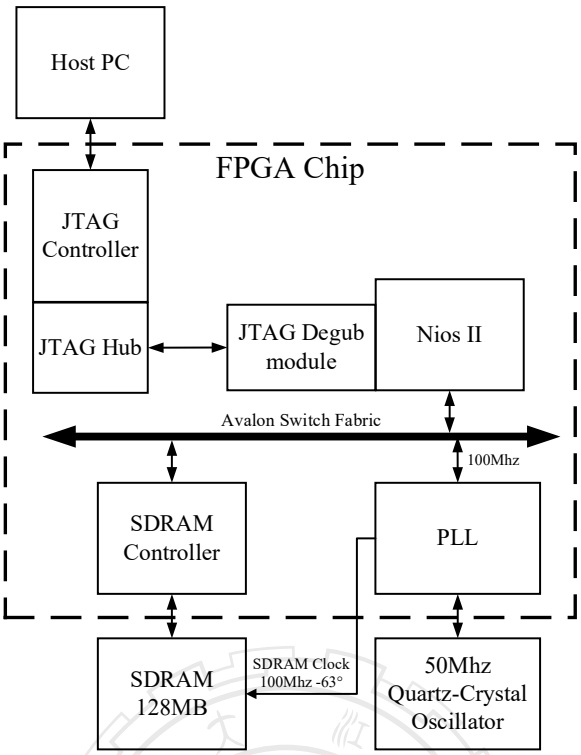


圖 5.19、程式下載器

在 Nios II Command Shell 中找到程式下載器的.sof 檔案，在此範例中命名為 Loader.sof，鍵入 nios2-configure-sof 即可完成配置。

如圖 5.20 所示，鍵入 nios2-configure-sof 後會得到一連串的 information，關於配置的開始時間還有結束時間以及 JTAG 的 ID 等等，最重要的訊息是要看是否配置成功，倒數第五行 Quartus II 32-bit Programmer was successful. 0 errors, 0 warnings 說明燒錄成功，0 錯誤，0 警告。

```
$ nios2-configure-sof
Searching for SOF file:
in .
   Loader.sof

Info: ************************************************************
Info: Running Quartus II 32-bit Programmer
Info: Command: quartus_pgm --no_banner --mode=jtag -o p;Loader.sof
Info (213045): Using programming cable "USB-Blaster [USB-0]"
Info (213011): Using programming file Loader.sof with checksum 0x006C049C for device
EP4CE115F29@1
Info (209060): Started Programmer operation at Thu Dec 29 03:33:21 2022
Info (209016): Configuring device index 1
Info (209017): Device 1 contains JTAG ID code 0x020F70DD
Info (209007): Configuration succeeded -- 1 device(s) configured
Info (209011): Successfully performed operation(s)
Info (209061): Ended Programmer operation at Thu Dec 29 03:33:27 2022
Info: Quartus II 32-bit Programmer was successful. 0 errors, 0 warnings
      Info: Peak virtual memory: 190 megabytes
      Info: Processing ended: Thu Dec 29 03:33:27 2022
      Info: Elapsed time: 00:00:07
      Info: Total CPU time (on all processors): 00:00:01
```

圖 5.20、配置 FPGA 成為程式下載器

現在 FPGA 已成為了程式下載器，再來就是將編譯好的測試程式下載到 SDRAM 中。

測試程式的程式碼如圖 5.21 所示，將 SDRAM 裡的資料以 4 Bytes 為單位讀出 20 筆資料，由於 elf 執行檔的機械碼也存放在 SDRAM 中，將打印出來的資料與 elf 執行檔進行比對即可知道程式是否正確下載到 SDRAM 中。

由於本實驗尚未將 stdout 重新導向 JTAG UART，因此無法使用 printf 函式，需透過 sprintf 函式將輸出文字存放到 buffer 字元陣列再交由自製的 print_str 函式將字串輸出。

```
#include<stdio.h>
#define SDRAM_BASE 0x08000000
#define JTAG_UART_BASE 0x000000C0
#define JTAG_UART_DATA *((int *)(JTAG_UART_BASE))
#define JTAG_UART_CTRL *((int *)(JTAG_UART_BASE + 4))
void print_str(char* p);
void _puts(char* p);
int main(void) {
	char buffer[30];
	for (int i = 0; i < 20; i++) {
		sprintf(buffer, "%08x:%08x\n", (int*)(SDRAM_BASE + 4 * i), *((int*)(SDRAM_BASE + 4 * i)));
		print_str(buffer);
	}
	return 0;
}
void print_str(char* p) {
	while (*p != '\0') {
		_puts(p++);
	}
}
void _puts(char* p) {
	while ((JTAG_UART_CTRL >> 16 & 0xFFFF) == 0);//no space to write
	JTAG_UART_DATA = *p;
}
```

圖 5.21、讀取 SDRAM 數值並以 JTAG UART 傳輸

此範例 elf 執行檔命名為 a.elf，在 Nios II Command Shell 中找到編譯好的 elf 執行檔後，鍵入 nios2-download a.elf 即可下載程式到 SDRAM 中，此工具會批次的將 elf 資料寫入 SDRAM 並讀回做驗證，若發現寫入和讀回的資料不匹配，會顯示驗證失敗等報錯。

如圖 5.22 所示，鍵入 nios2-download a.elf 後會看到程式大小 79KB、下載程式花了 0.9 秒等資訊，最後看到 Verified OK 代表成功下載。

```
$ nios2-download    a.elf
Using cable "USB-Blaster [USB-0]", device 1, instance 0x00
Pausing target processor: OK
Initializing CPU cache (if present)
OK
Downloaded 79KB in 0.9s (87.7KB/s)
Verified OK
Leaving target processor paused
```

圖 5.22、下載 a.elf 執行檔到 SDRAM

現在 RISC-V 程式已經正確下載到 SDRAM 了，剩下的就是把 FPGA 重新配置成我們的目標系統，讓目標系統去執行程式。

目標系統架構如圖 5.23 所示，跟圖 5.12、JTAG UART Demo 系統架構稍微不一樣，移除了 On-Chip Memory，增加了 SDRAM, SDRAM Controller 和 PLL 等元件。

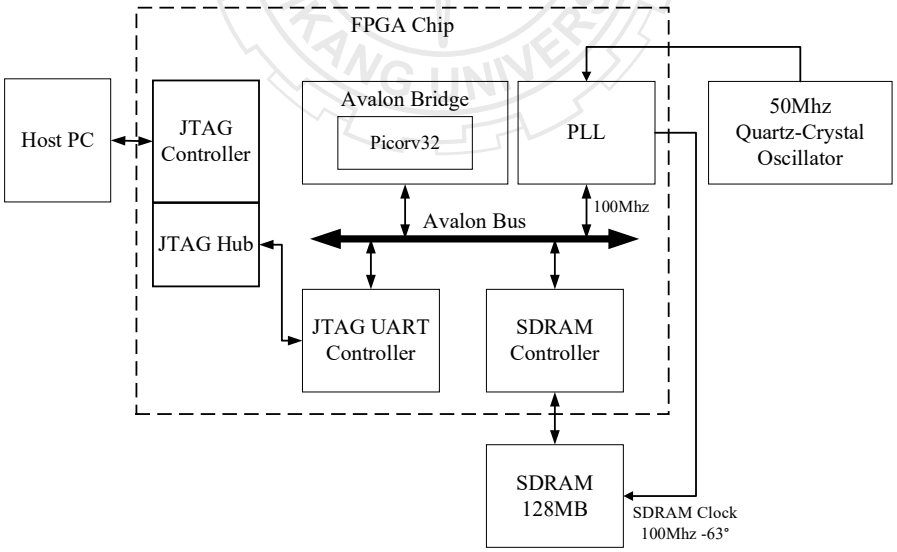


圖 5.23、目標系統架構

由於 SDRAM 的時序比較複雜，需要增加 SDRAM 控制器來處理時序問題，其 SDRAM 控制器參數如圖 5.24 和圖 5.25 所示，又因 SDRAM 的時脈與系統時脈有相位差，因此系統中還增加了鎖相迴路(Phase-locked loop, PLL)元件來幫助產生相位差和倍頻，其 PLL 參數如圖 5.26 和圖 5.27 所示，c0 為系統時脈，c1 為 SDRAM 時脈與 c0 有-63 度的相位差，c0 和 c1 時脈均操作在 100Mhz。

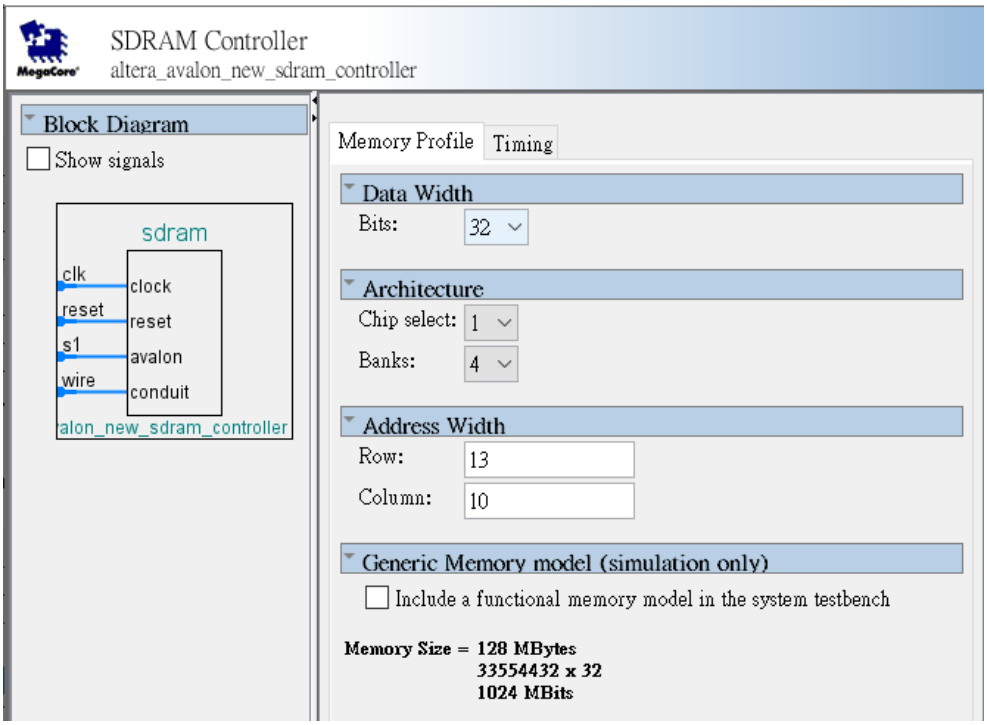


圖 5.24、設定 SDRAM 大小

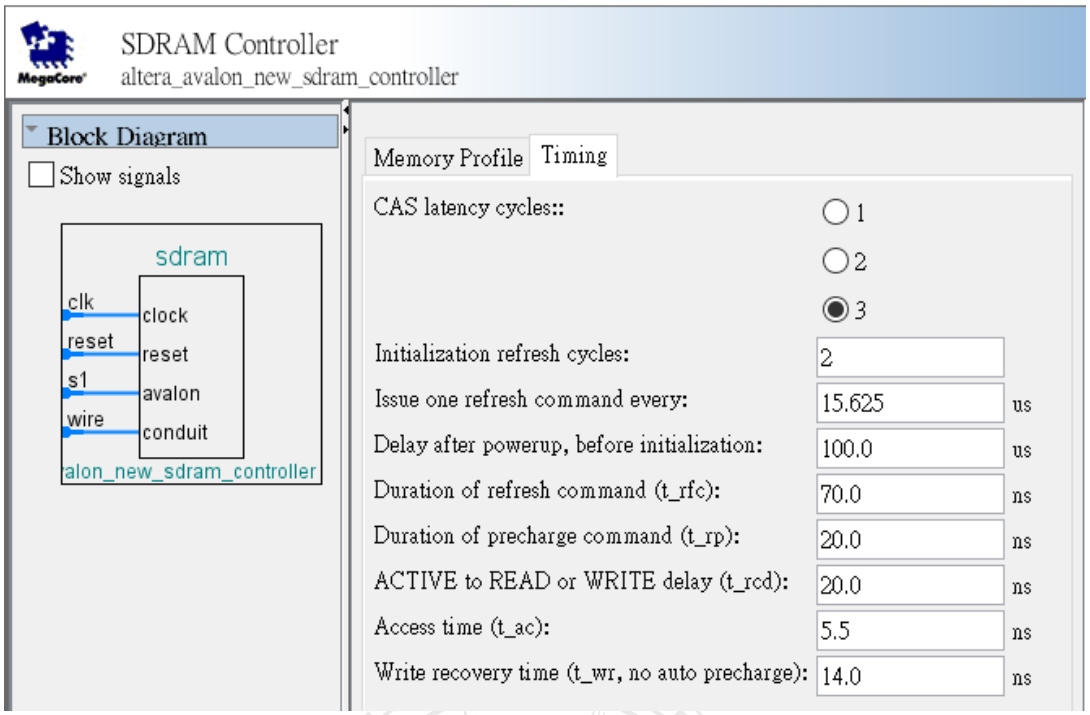


圖 5.25、設定 SDRAM 時序控制參數

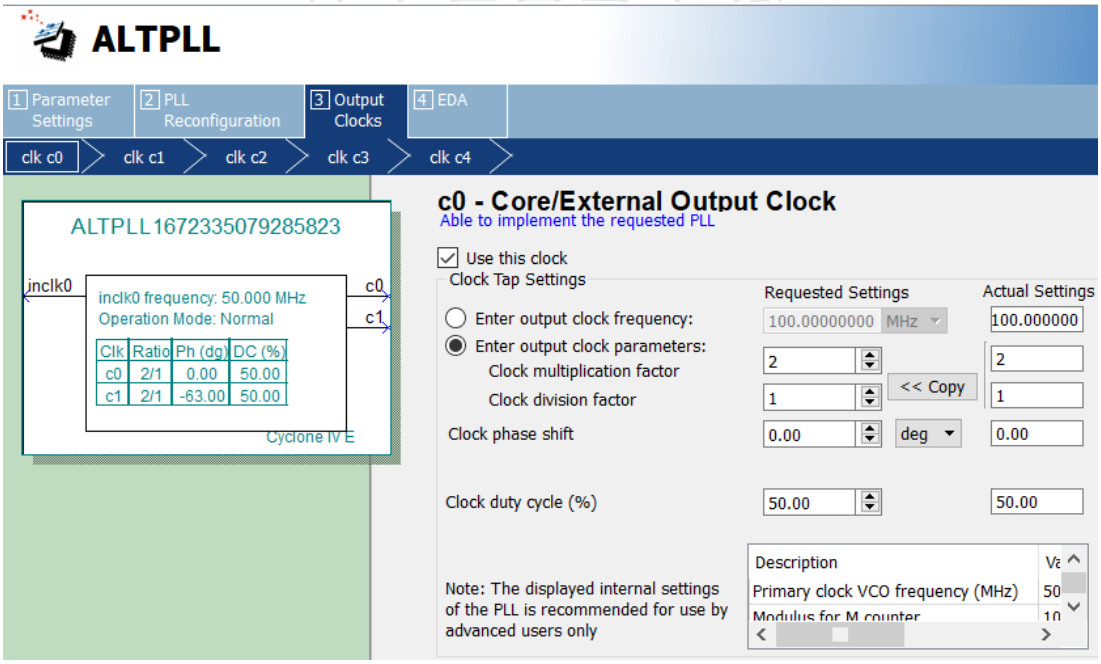


圖 5.26、設定系統時脈 c0

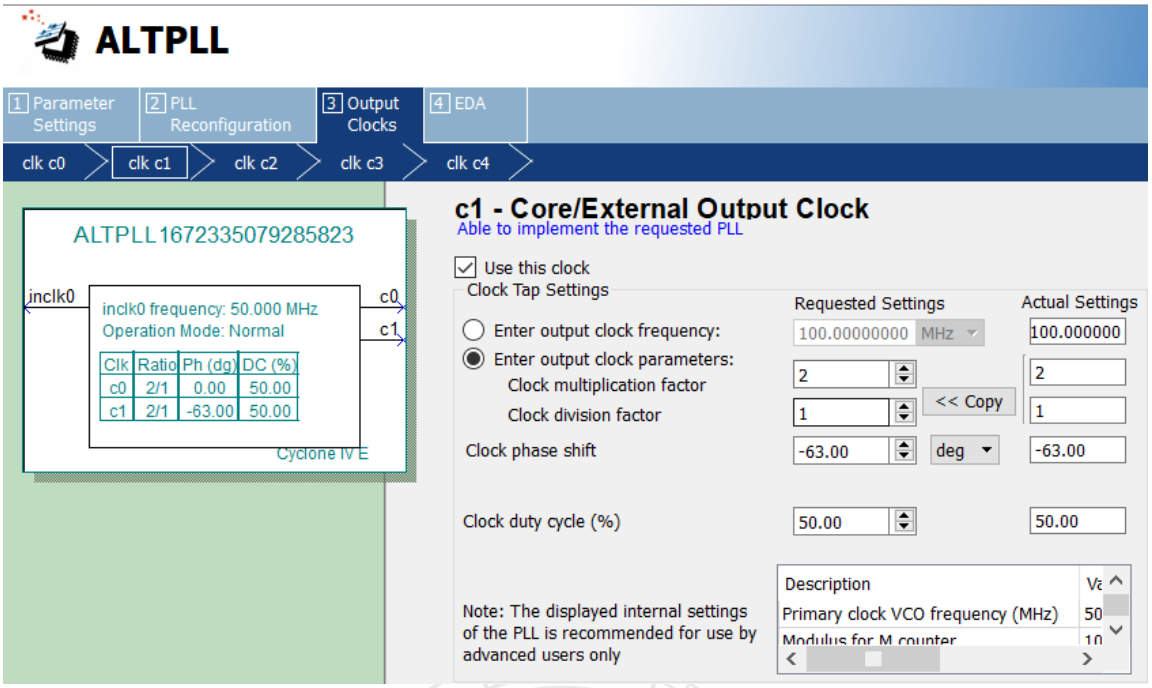


圖 5.27、設定 SDRAM 時脈 c1

此範例的目標系統的.sof 檔案命名為 Target.sof，在 Nios II Command Shell 中找到目標系統 Target.sof 後，鍵入 nios2-configure-sof 配置 FPGA 成為我們的目標系統，就可以正常運行了。

完整的通訊架構如圖 5.28 所示，MAXII CPLD 和 FT245 USB FIFO 統稱為 USB-Blaster 電路，JTAG UART 資料會經由 USB-Blaster 電路連接到 Host PC，透過 nios2-terminal 工具來顯示接收到的資料。

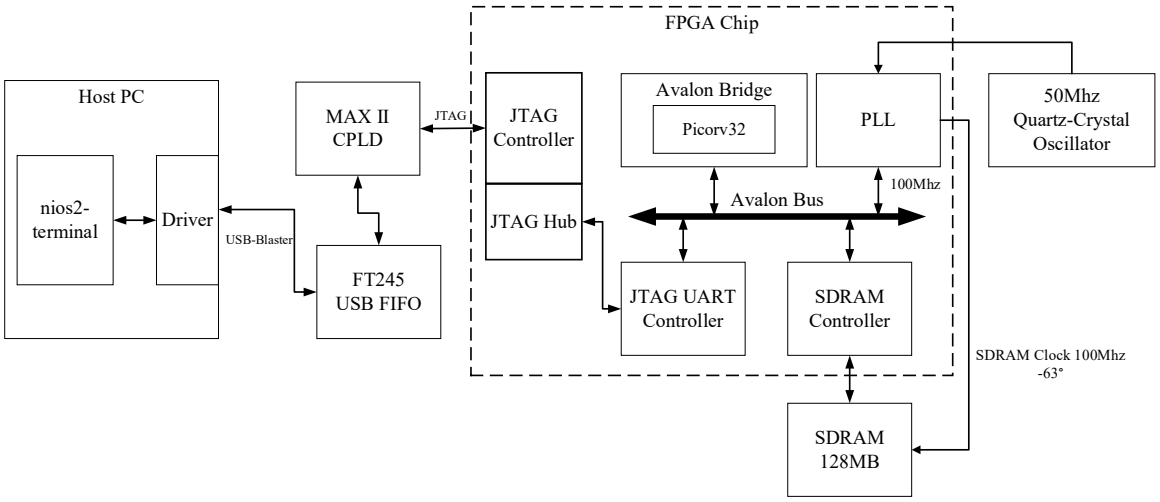


圖 5.28、目標系統的通訊架構

如圖 5.29 所示，鍵入 nios2-configure-sof 後確認 Info 資訊，顯示配置成功。

```
$ nios2-configure-sof
Searching for SOF file:
in .
   Target.sof

Info: ************************************************************
Info: Running Quartus II 32-bit Programmer
Info: Command: quartus_pgm --no_banner --mode=jtag -o p;Target.sof
Info (213045): Using programming cable "USB-Blaster [USB-0]"
Info (213011): Using programming file Target.sof with checksum 0x00850371 for device EP4CE115F29@1
Info (209060): Started Programmer operation at Thu Dec 29 03:33:36 2022
Info (209016): Configuring device index 1
Info (209017): Device 1 contains JTAG ID code 0x020F70DD
Info (209007): Configuration succeeded -- 1 device(s) configured
Info (209011): Successfully performed operation(s)
Info (209061): Ended Programmer operation at Thu Dec 29 03:33:42 2022
Info: Quartus II 32-bit Programmer was successful. 0 errors, 0 warnings
      Info: Peak virtual memory: 190 megabytes
      Info: Processing ended: Thu Dec 29 03:33:42 2022
      Info: Elapsed time: 00:00:07
      Info: Total CPU time (on all processors): 00:00:01
```

圖 5.29、配置 FPGA 成為目標系統

在 Nios II Command Shell 中鍵入 nios2-terminal 打開終端介面，如圖 5.30 所示，如有正常連接將會打印出從 JTAG UART 傳來的資料。

```
$ nios2-terminal
nios2-terminal: connected to hardware target using JTAG UART on cable
nios2-terminal: "USB-Blaster [USB-0]", device 1, instance 0
nios2-terminal: (Use the IDE stop button or Ctrl-C to terminate)

08000000:00014197
08000004:1f818193
08000008:88818513
0800000c:88818613
08000010:40a60633
08000014:00000593
08000018:1c0000ef
0800001c:0000e517
08000020:00450513
08000024:00050863
08000028:00008517
0800002c:d1c50513
08000030:7f10d0ef
08000034:284050ef
08000038:00012503
0800003c:00410593
08000040:00000613
08000044:06c000ef
08000048:2280506f
0800004c:ff010113
```

圖 5.30、nios2-terminal 顯示 JTAG UART 資料

使用 gcc 的 objdump 工具將 a.elf 執行檔 dump 出來如圖 5.31 所示，和圖 5.30 中 nios2-terminal 打印的資料做比對，確認程式成功地下載到 SDRAM 中。

```
Disassembly of section .text:


08000000 <_start>:
  8000000: 00014197                auipcgp,0x14
  8000004: 1f818193                addi  gp,gp,504 # 80141f8 <__global_pointer$>
  8000008: 88818513                addi  a0,gp,-1912 # 8013a80 <_etext>
  800000c: 88818613                addi  a2,gp,-1912 # 8013a80 <_etext>
  8000010: 40a60633                sub   a2,a2,a0
  8000014: 00000593                li    a1,0
  8000018: 1c0000ef          jal   ra,80001d8 <memset>
  800001c: 0000e517                auipca0,0xe
  8000020: 00450513                addi  a0,a0,4 # 800e020 <atexit>
  8000024: 00050863                beqz a0,8000034 <_start+0x34>
  8000028: 00008517                auipca0,0x8
  800002c: d1c50513                addi  a0,a0,-740 # 8007d44 <__libc_fini_array>
  8000030: 7f10d0ef          jal   ra,800e020 <atexit>
  8000034: 284050ef                jal   ra,80052b8 <__libc_init_array>
  8000038: 00012503                lw    a0,0(sp)
  800003c: 00410593                addi  a1,sp,4
  8000040: 00000613                li    a2,0
  8000044: 06c000ef          jal   ra,80000b0 <main>
  8000048: 2280506f                j     8005270 <exit>

0800004c <__do_global_dtors_aux>:
  800004c: ff010113          addi  sp,sp,-16
```

圖 5.31、objdump a.elf 執行檔

5.4 處理器資源消耗和性能比較

針對 Picorv32 處理器、低階版 Nios II/e 和高階版 Nios II/f 三者處理器做出資源消耗和性能的比較，資源消耗比較如表 5.3 所示，Picorv32 是

三者中總體資源消耗最少的處理器，雖然 LE(Logic Element)數量看起來會比 Nios II/e 處理器多一些，但是 Memory bits 卻少了很多，這是因為 Nios II/e 處理器的部分設計使用了 FPGA 晶片中的 On-chip Memory 而不是使用 LE 直接合成，因而減少了 LE 的數量；處理器性能比較如表 5.4 所示，Picorv32 處理器在 DMIPS 和 Coremark 的性能跑分測試中[21]，其性能表現介於三者之中，而在系統可運行的最大頻率 Fmax 以 Picorv32 處理器最高。

Nios II 處理器的性能表現參考 Intel 官方的 Performance Benchmarks 文件[22]。

表 5.3、處理器資源消耗比較

Processor	Picorv32	Nios II/e	Nios II/f
LC(Logic Cell) Combinationals	1593	995	2358
LC(Logic Cell) Registers	649	662	1789
Total LE	1679	1136	3015
DSP Elements	0	0	4
Memory Bits	2048	10240	64448
Fmax Mhz	138.12	107.5	125.6
FPGA Device	Cyclone IV EP4CE115		

表 5.4、處理器性能比較

Processor	Picorv32	Nios II/e	Nios II/f
DMIPS/Mhz	0.527	0.107	0.753
CoreMark/Mhz	≈0.4	0.192	2.291

第六章 結論與未來展望

6.1 結論

本論文的實驗橋接了 Picorv32 處理器至 Avalon 系統匯流排，並進行了多種的實驗展示，實驗結果顯示其功能均正確，完整地示範了軟硬體的設計與操作流程，從一開始處理器的選用和參數修改、橋接器的設計與橋接過程、如何打造 SOPC、程式的編譯和下載，到最後讓系統運行起來，給後繼的研究者提供了一個方向，一個可行的搭建流程。

本論文的實驗架構相比於在緒論中提到的友晶科技 T-Core e203 RISC-V SoC 解決方案得到了多處改進，本來需要使用三個作業系統來實現硬體的合成、軟體的編譯、程式的下載，現僅需要在 Windows 10 或是 Linux 任一作業系統即可完成，Quartus II、RISC-V 編譯器工具鏈和 Nios II EDS 在 Windows 及 Linux 作業系統中都有提供相對應的版本；相比於 e203 SoC 採用私有的周邊匯流排架構，使用者不易新增周邊，本論文實驗的 Picorv32 SoC 採用 Avalon 匯流排架構，可以輕鬆添加 PIO、JTAG UART、SDRAM Controller 等眾多周邊；相比於 T-Core 在硬體上需要兩組 JTAG 和一組 UART 來實現配置 FPGA、下載 RISC-V 程式以及 UART 傳輸，本論文使用 Nios II Debug module 來幫助下載程式僅需一組 JTAG 且，減少了指撥開關和 USB to TTL 轉接器的硬體成本，開發流程上更為簡單快速，開發者不須在三個作業系統中來回切換，更不用切換指撥開關來決定要配置 FPGA 還是下載 RISC-V 程式；RISC-V 程式下載速度也得到改進，在實驗結果中顯示使用 Nios II 的 Debug module 下載 RISC-V 程式僅需 0.9 秒，相比於友晶科技在 VirtualBox 虛擬機使用自製的下載工具快了將近五十倍，大大地節省了開發的時間成本。

Avalon 架構簡單有彈性，透過 Qsys 工具可以輕鬆的添加和修改周邊裝置，大大的降低了 SOPC 的設計門檻，以往以 Avalon 介面所開發的 IP 也得以繼續沿用，在開發嵌入式系統上省去很多不必要的麻煩。

Picorv32 處理器雖在性能表現尚不及 Nios II/f 處理器，但替換掉 Nios II/e 處理器卻綽綽有餘，使用者將不再受限於 Nios II/e 處理器的低性能以及 Nios II/f 處理器的授權限制，可以使用本論文設計的系統進行程式開發，運用在低功耗嵌入式系統上，亦能替換成更高性能的處理器，面對不同的需求能有更彈性的選擇。

6.2 未來展望

本論文尚有許多可以增加的實驗，如增加客製化指令集、壓縮指令集、致能中斷功能、致能協處理器使用硬體乘除法器、FreeRTOS 作業系統移植等，更希望能夠透過以上的實驗將其內容融合到大學部的課程，配合基礎學科如邏輯設計、計算機組織、作業系統，將有機會帶領學生從邏輯閘開始慢慢堆砌，到最後做出一個能夠運行作業系統的嵌入式系統。

希望在未來能夠整合性能更強的 RISC-V 處理器核心，搭配 AI(Artificial Intelligence)加速器等周邊 IP，應用在人工智慧邊緣運算領域，也希望能自行設計高階處理器，走向高端市場應用。

參考文獻

[1] 聶建中，美國晶片法案下的科技新冷戰及兩岸半導體業發展，展望與探索，第 20 卷，頁 16，2022 年 9 月。

[2] e203_hbirdv2:The Ultra-Low Power RISC-V Core, URL: https://github.com/riscv-mcu/e203_hbirdv2

[3] Altera DE2-115 教育開發平台, URL:https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=Taiwan&CategoryNo=171&No=542

[4] RISC-V GNU Compiler Toolchain, URL：https://github.com/riscv-collab/riscv-gnu-toolchain

[5] X. Xing, C. Zezong, J. Jing and K. Hengyu, "Porting from Wishbone Bus to Avalon in SoC Design," *International Conference on Electronic Measurement and Instruments, Xi'an China*, pp.1-862~1-865, 2007.

[6] *Avalon Interface Specifications*, Intel inc., pp.21-22, 2021.

[7] Rocket Chip Generator - Github, URL：https://github.com/chipsalliance/rocket-chip

[8] Source file for Sifive's Freedom platforms - Github, URL：https://github.com/sifive/freedom

[9] *The RISC-V Instruction Set Manual Volume I: Unprivileged ISA*, SiFive inc., 2019.

[10] PicoRV32 - A Size-Optimized RISC-V CPU - GitHub, URL：https://github.com/YosysHQ/picorv32

[11] *Platform Designer User Guide*, Intel inc., pp.319-320, 2018.

[12] PulseRain Reindeer - RISC-V RV32I[M] Soft CPU, URL：https://github.com/PulseRain/Reindeer

[13] David A. Patterson., John L. Hennessy. *Computer Organization and*

Design RISC-V Edition, 2ed. San Francisco: Morgan Kaufmann, ISBN:978-0-12-820331-6, pp.131-134, 2020.

[14] *Using the GNU Compiler Collection*, Richard M. Stallman, GCC Developer Community, pp.428-431.

[15] *Intel Quartus Prime Pro Edition*, Intel inc., pp.73-74, 2021.

[16] *The GNU Binary Utilities*, Roland H. Pesch, Jeffrey M. Osier, Cygnus Support, pp.1-82, 2010.

[17] John R. Levine. *Linkers & Loaders*, 1st. San Francisco: Morgan Kaufmann, pp.84-91, 1999.

[18] *Intel Hexadecimal Object File Format Specification Revision A*, Intel inc., pp.2-4, 1988.

[19] *Nios II Classic Software Developer's Handbook*, Intel inc., pp.7, 57-58, 2015.

[20] *Embedded Peripherals IP User Guide*, Intel inc., pp.149-157, 2020.

[21] C. Heinz, Y. Lavan, J. Hofmann and A. Koch, "A Catalog and In-Hardware Evaluation of Open-Source Drop-In Compatible RISC-V Softcore Processors," *International Conference on ReConFigurable Computing and FPGAs (ReConFig), Cancun, Mexico*, pp.1-8, 2019.

[22] *Nios II Performance Benchmarks*, Intel inc., pp.1-8, 2020.